

Processador Mancha

Marcelo Pasin

26 de Agosto de 2004

Conteúdo

1	Introdução	2
2	Modos de Operação	3
3	Conjunto de Registradores	4
4	Modos de Endereçamento	6
5	Conjunto de Instruções	10
6	Formato das Instruções	18
7	Interrupções	22
8	Organização da Memória	24
9	Resumo do Processador Mancha	28
10	Montador	30
11	Programas Utilitários	37
12	Formato do arquivo objeto	39
13	Simulador	43
14	Mancha Mínimo	47

1 Introdução

Compreender o funcionamento dos computadores modernos, por estarem cada dia mais complexos, é uma tarefa difícil para um estudante principiante em informática. Em especial, um estudante principiante desconhece a maioria dos conceitos de arquitetura e organização de computadores. O processador Mancha foi concebido com o objetivo de facilitar o ensino de arquitetura de computadores. Ele é um processador bastante simples e didático que possui a maioria das características de um processador moderno. Para ele foram desenvolvidas diversas ferramentas, como um simulador, um montador e um ligador (está sendo desenvolvido até mesmo um compilador C), de modo a permitir a construção e a execução de programas. Isto permite desenvolver trabalhos práticos bastante realistas para disciplinas como arquitetura e organização de computadores e sistemas operacionais.

Este texto tem o objetivo de descrever o processador Mancha e as ferramentas para ele desenvolvidas.

Modelo Completo e Modelo Mínimo

O Mancha é um processador hipotético de 16 bits de propósito geral, concebido inicialmente como ferramenta didática da disciplina de Programação de Sistemas do Curso de Informática da Universidade Federal de Santa Maria (hoje Ciência da Computação). Ele possui dois modelos, um completo e outro reduzido (mínimo). O modelo completo implementa a maioria das características de um processador moderno, mantendo níveis baixos de complexidade, de forma a permitir a sua inteira compreensão por um aluno principiante no assunto.

O modelo reduzido, chamado Mancha Mínimo, implementa um subconjunto das instruções disponíveis no modelo completo. Geralmente este modelo é usado em disciplinas de organização de computadores, onde em aulas práticas se faz a implementação de um pequeno processador usando blocos funcionais digitais. O processador completo executa todas as instruções do modelo reduzido, permitindo assim utilizar-se o mesmo conjunto de ferramentas em mais de uma disciplina.

2 Modos de Operação

Como todo processador moderno, o Mancha possui dois modos de operação: modo usuário e modo supervisor. No modo usuário de operação, o processador não pode executar todas as instruções de seu conjunto, nem pode acessar toda a memória disponível. Este modo foi criado para permitir a execução segura de instruções de programas escritos por usuários do computador. O modo supervisor, no qual o processador pode executar qualquer instrução definida em seu conjunto, serve para executar o programa do sistema operacional.

A distinção entre usuário e supervisor serve para proteger o sistema do mau uso por programas de usuário. Se um destes programas estiver errado e indevidamente modificar conteúdos de memória ou manipular dispositivos de entrada e saída, este pode vir a prejudicar os outros programas que estão sendo executados. Assim, em um computador moderno, diversos programas podem estar em execução, cada um deles operando sem interferir nos outros.

O único programa que pode interferir no funcionamento dos outros é o sistema operacional, já que é essa a sua função. Todas as operações que podem por em risco o funcionamento do computador são feitas pelo sistema operacional (ainda que por pedido dos programas de usuário). Considera-se que o sistema operacional não vai operar indevidamente e, por isso, não vai prejudicar a operação dos outros programas. Claro que isso não evita que uma certa empresa continue vendendo seus sistemas operacionais com erros...

3 Conjunto de Registradores

A vasta maioria dos processadores existentes possui em seu interior um conjunto de registradores. Este conjunto serve como depósito para dados dentro do processador, enquanto estes dados estão sendo operados. Os processadores têm instruções para carregar (*load*) dados da memória e guardá-los em registradores internos, armazenar (*store*) dados de registradores na memória, bem como instruções para fazer operações com os dados contidos em registradores.

Por exemplo, suponha que um programa deve calcular o lucro de uma empresa subtraindo o total de sua despesa do total de sua receita. Tanto receita e despesa são números que estão guardados em memória. O programa que calcula o lucro contém duas instruções para carregar registradores com os valores de receita e despesa. Após a carga, uma operação de subtração é feita, deixando o resultado em um registrador qualquer. Enfim, uma operação de armazenamento é necessária para enviar o resultado da operação à memória, guardando o valor do lucro.

O Mancha possui 16 registradores, oito de usuário e oito de supervisor, todos com 2 bytes (16 bits). Os registradores de supervisor somente podem ser acessados em modo supervisor. Os seis primeiros registradores de usuário têm propósito geral, sendo o sexto registrador geralmente usado como ponteiro de base de alocação automática. O sétimo registrador (r6) serve como ponteiro da pilha do processador e o oitavo registrador (r7) serve como ponteiro de instruções. Os registradores de supervisor têm, cada um, uma função específica. Estas funções serão explicadas em outras seções do texto. A figura 1 mostra os registradores do Mancha. Nesta figura aparecem os nomes dos registradores, os códigos binários correspondentes e a descrição de cada um.

Os registradores de r0 a r5 podem ser usados em qualquer operação de acesso a memória, de aritmética ou de lógica. São chamados registradores de usuário de **propósito geral**. O registrador bp, o ponteiro de base (*base pointer*), é também um registrador de propósito geral. Ele é usado tradicionalmente em programas compilados, que implementem chamadas de procedimentos. Nestes programas, o bp é usado como **ponteiro do quadro** de ativação, que contém as variáveis locais do procedimento sendo executado.

Os registradores sp e ip são registradores especiais. Existem certas instruções que fazem uso específico destes registradores, sem nem sequer mencioná-los. Eles podem ainda assim ser usados em quaisquer operações de acesso a memória, de aritmética ou de lógica, como os outros seis registradores. O registrador ip é o **ponteiro de instruções** do processador (*instruction pointer*), também chamado de contador de programa. O número nele contido é usado como endereço de busca para instruções na memória. A cada busca ele é incrementado, permitindo assim que o processador execute programas compostos por instruções sequencialmente dispostas na memória.

O registrador sp é o **ponteiro da pilha** (*stack pointer*) do processador. Quando um procedimento é chamado, o valor do contador de programa é guardado na memória, na posição dada pelo registrador sp. O ip recebe então um novo valor, correspondente à primeira instrução do procedimento sendo chamado. Para retornar à posição original, que estava sendo executada antes da chamada, o processador carrega o registrador ip da posição de memória indicada em sp. De forma a poder chamar diversos procedimentos, a cada chamada o valor de sp é decrementado, sendo novamente incrementado no retorno. O sp funciona assim como um ponteiro de uma pilha de valores de ip.

O Mancha possui um **registrador de estado**, sr. Este registrador contém bits que sinalizam di-

Figura 1 Conjunto de registradores do Mancha



versas condições. Se alguma destas condições for verdadeira, o bit correspondente estará ligado. As condições sinalizadas no registrador *sr* são:

- S: modo supervisor ativado
- P: paginação ativada
- I: processador pode usar instruções IN e OUT
- D: interrupções desativadas
- N: o resultado da última operação é negativo
- O: o resultado da última operação não cabe em 16 bits(*overflow/underflow*)
- Z: o resultado da última operação é zero
- C: bit de transporte, “vai um” (*carry*)

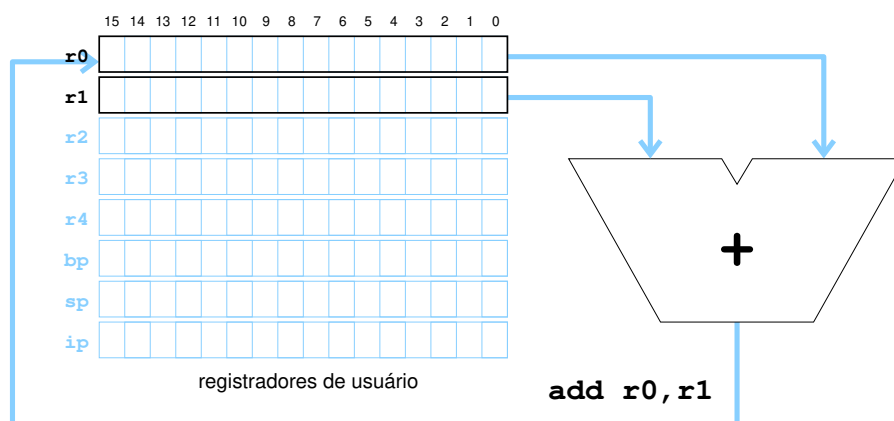
4 Modos de Endereçamento

Diversas são as formas de se especificar qual será o dado a ser usado em uma operação aritmética ou lógica. Por exemplo, um programa poderia somar A com B e guardar o resultado em C , depois somar 2 com o valor guardado em C . A , B e C são variáveis que estão em memória. Usar seus valores significa usar dados que estão na memória. Já 2 é uma constante e não precisa necessariamente estar guardada em uma variável, pois pode fazer parte da instrução.

Para implementar acessos a dados, o processador Mancha possui sete modos de endereçamento. Estes modos são designados por um código de 3 bits. Em geral os modos de endereçamento estão associados a um registrador de usuário. O texto a seguir descreve estes modos de endereçamento e um resumo dos modos pode ser visto em na tabela 1.

Modo direto (000) O dado é o conteúdo de um registrador. No exemplo da figura 2, o conteúdo do registrador $r1$ é adicionado em $r0$. Este modo de endereçamento serve para efetuar operações com valores que já se encontram em algum dos registradores internos do processador.

Figura 2 Modo de endereçamento: direto



Modo indireto (001) O dado é uma palavra de memória apontada por um registrador de índice. No exemplo da figura 2, $r2$ recebe conteúdo da memória apontada por $r3$. Este modo de endereçamento serve para acessar dados com posição variável na memória, como no caso de ponteiros por exemplo. O registrador de índice funciona como um ponteeiro para a memória.

Modo pós-incremento (010) O dado é uma palavra apontada por um registrador de índice, que é incrementado após o acesso. Se a operação for de uma palavra, o registrador de índice é incrementado de dois, se for byte, de um. No exemplo da figura 4, $r3$ é carregado do endereço apontado por sp , incrementando sp após. Esta operação também é conhecida como desempilhar (*pop*). Este modo de endereçamento é útil para implementar pilhas e para percorrer vetores de dados em memória.

Modo pré-decremento (011) O dado é uma palavra apontada por um registrador de índice, que é decrementado antes do acesso. Se a operação for de uma palavra, o registrador de índice é decrementado de dois, se for byte, de um. No exemplo da figura 5, $r1$ é armazenado na memória apontada por sp que é previamente decrementado. Esta operação também é conhecida como empilhar (*push*). Este modo de endereçamento é complementar ao modo de pós-incremento, sendo também útil para implementar pilhas e para percorrer vetores de dados em memória.

Figura 3 Modo de endereçamento: indireto

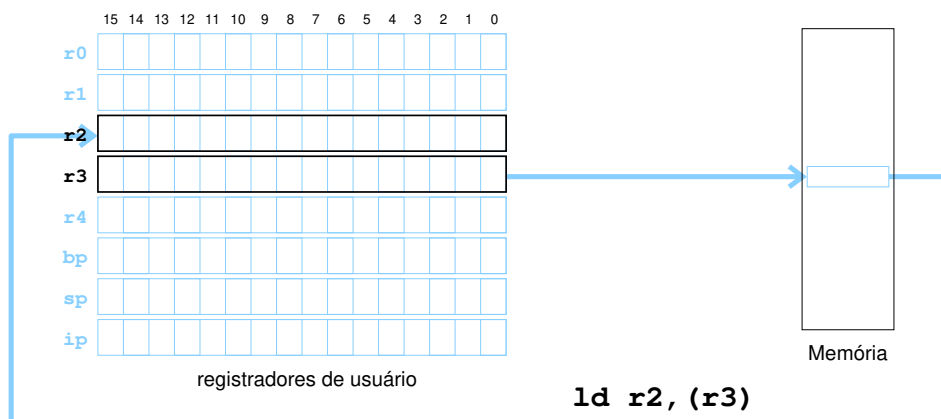
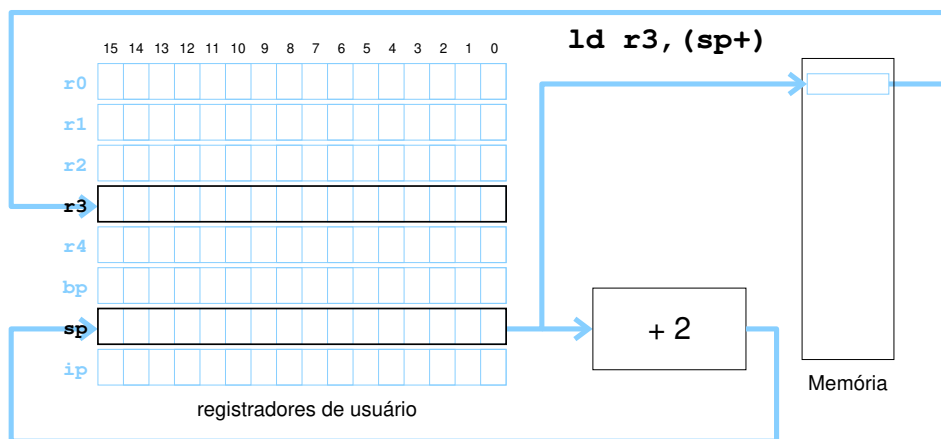


Figura 4 Modo de endereçamento: pós-incremento



Modo imediato (100) O dado é uma palavra armazenada imediatamente após a instrução. Este modo de endereçamento causa o acréscimo de uma palavra à instrução, chamada de valor imediato. O exemplo da figura 6 subtrai o valor constante 5 de r1. Este modo de endereçamento serve para inserir valores constantes em uma expressão.

Figura 5 Modo de endereçamento: pré-decremento

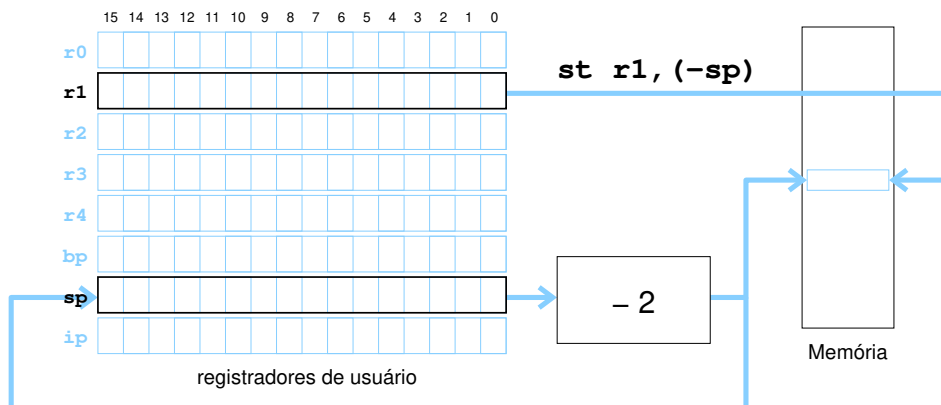
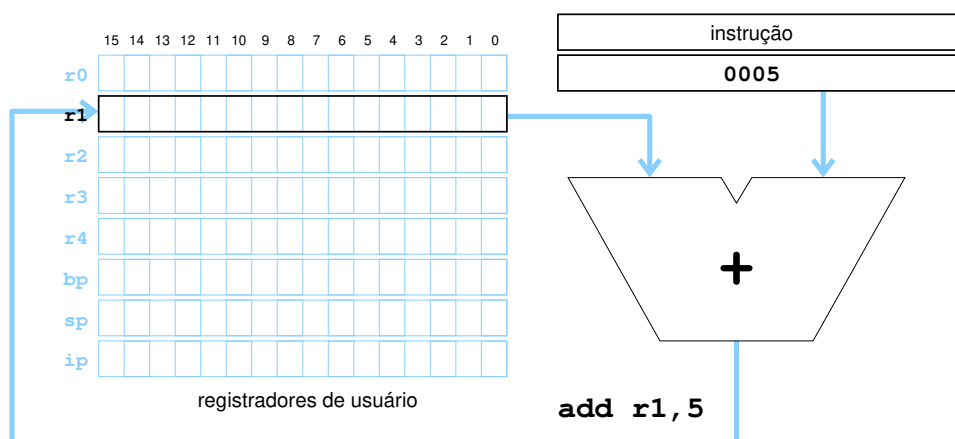
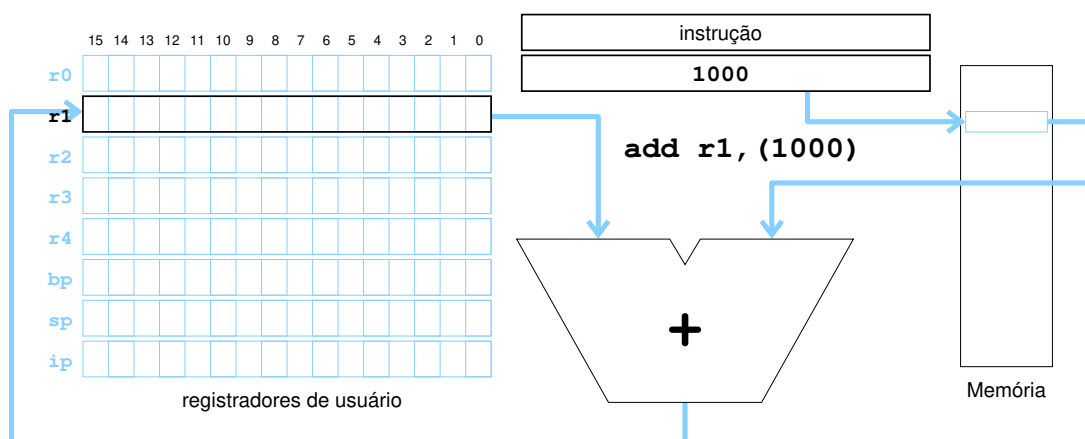


Figura 6 Modo de endereçamento: imediato



Modo absoluto (101) O dado é a palavra apontada pela palavra armazenada logo após a instrução. Este modo de endereçamento também causa o acréscimo de uma palavra à instrução (valor imediato). O exemplo da figura 7 adiciona em *r1* a palavra armazenada na memória a partir do endereço 1000. Este modo de endereçamento serve para acessar dados em posições fixas da memória, como é o caso de variáveis globais ou estáticas de um programa.

Figura 7 Modo de endereçamento: absoluto



Modo deslocamento (110) O dado é uma palavra armazenada na memória, cujo endereço se calcula somando a palavra armazenada imediatamente após a instrução ao conteúdo de um registrador de índice. Este modo de endereçamento também causa o acréscimo de uma palavra à instrução (valor imediato). O exemplo da figura 8 carrega em *bp* a palavra armazenada na memória a partir do endereço dado por *r1* somado com 10. Este modo de endereçamento serve para implementar acessos a dados armazenados com deslocamentos fixos em relação a uma dada base. Exemplos de dados com deslocamentos fixos são os campos de uma estrutura de dados dinamicamente alocada, ou as variáveis locais de um procedimento.

Figura 8 Modo de endereçamento: deslocamento

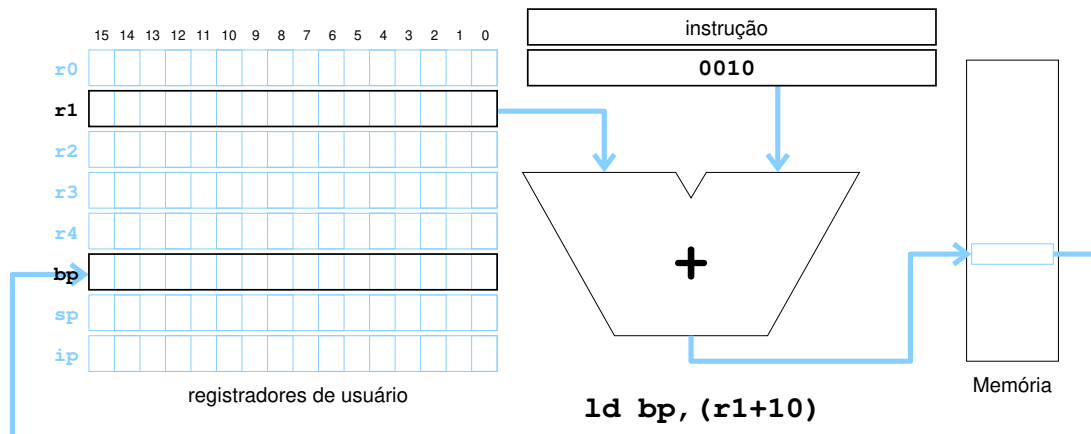


Tabela 1 Resumo dos modos de endereçamento

nome	número	exemplo
direto	000	ld r0, r2
indireto	001	ld r0, (r2)
pós-incremento	010	ld r0, (r2+)
pré-decremento	011	ld r0, (-r2)
imediato	100	ld r0, 0002
absoluto	101	ld r0, (0002)
deslocamento	110	ld r0, (r1+0002)

5 Conjunto de Instruções

O processador Mancha possui instruções para movimentação de dados, controle da execução, operações aritméticas e operações lógicas. Estas instruções possuem argumentos que especificam os seus dados, usando o conjunto de registradores descritos na seção 3 e os modos de endereçamento descritos na seção 4. As instruções descritas a seguir utilizam a seguinte notação: o texto em tamanho *fixo* designa um nome de instrução e o texto em *itálico* designam uma opção com vários valores válidos. As opções existentes são:

- *REG*: um registrador de usuário. Valores válidos: *r0* a *r7*, *bp*, *sp* e *ip*;
- *SUP*: um registrador de supervisor. Valores válidos: *s0* a *s7*, *sr*, *cs*, *cl*, *ds* e *dl*;
- *IMED*: um valor imediato de 16 bits;
- *DATA*: um dado em um dos modos de endereçamento. Valores válidos: *REG*, (*REG*) (*REG*+), (*-REG*), *IMED*, (*IMED*) e (*REG*+*IMED*);
- *IM₁₀*: um valor de 10 bits com sinal ($-512 \leq IM_{10} \leq 511$);
- *IM₆*: um valor de 6 bits com sinal ($-32 \leq IM_6 \leq 31$);
- *COND*: uma condição. Valores válidos: *Z*, *EQ*, *C*, *LO*, *O*, *N*, *NP*, *NZ*, *NE*, *NC*, *HS*, *NO*, *NN*, *P*, *GE*, *LT*, *GT*, *LE*, *HI* e *LS*.

Instruções de Movimentação de Dados

Carga de registradores

```
ld    REG, DATA
ldb   REG, DATA
lds   SUP, DATA
ldq   REG, IM10
```

Carregam valor *DATA* ou *IM₁₀* em um registrador (*REG* ou *SUP*). As condições *N* e *Z* são sinalizadas de acordo com o valor carregado. *ld* (*load*), *ldb* (*load byte*) e *lds* (*load supervisor*) podem usar todos os modos de endereçamento em *DATA*. *ldb* opera somente com os 8 bits menos significativos de *DATA* e *REG*. *lds* carrega registradores do supervisor e é uma instrução privilegiada, ela somente pode ser usada se a condição *S* estiver ligada. *ldq* (*load quick*) aceita somente um valor imediato *IM₁₀*, de 10 bits, fazendo a extensão do seu sinal. Exemplos:

```
ld    r1, 5           ldb   r2, (r3+4)
lds   ds, 0           ldq   r0, -7
```

Armazenamento de registradores

```
st    REG, DATA
stb   REG, DATA
sts   SUP, DATA
```

Armazena valor de *REG* ou *SUP* em *DATA*. Nenhuma condição é modificada. *st* (*store*), *stb* (*store byte*) e *sts* (*store supervisor*) podem usar todos os modos de endereçamento em *DATA*, exceto o imediato. *stb* opera somente com os 8 bits menos significativos de *DATA* e *REG*. *sts* armazena registradores do supervisor mas não é uma instrução privilegiada. Exemplo:

```
st    r1, (r2+)
```

Salva registradores na pilha

```
push  REG
pushs SUP
```

Armazena registrador *REG* ou *SUP* na pilha do processador. O ponteiro da pilha (*sp*) é decrementado de dois antes de armazenar o valor na memória. Usada em conjunto com a instrução *pop*. Nenhuma condição é modificada. *pushs* salva registradores do supervisor mas não é uma instrução privilegiada. Exemplo:

```
push  r1
```

Recupera registradores da pilha

```
pop  REG
pops SUP
```

Recupera registrador *REG* ou *SUP* da pilha do processador. O ponteiro da pilha (*sp*) é incrementado de dois depois de recuperar o valor da memória. Usado em conjunto com *push*. As condições *N* e *Z* são sinalizadas de acordo com o valor recuperado. *pops* é uma instrução privilegiada. Exemplo:

```
pop  bp
```

Entrada de dados

```
in    REG, DATA
inb   REG, DATA
```

Lê dados (*input*) fornecidos pelo dispositivo conectado à porta *DATA*, armazenando-os em *REG*. As condições *N* e *Z* são sinalizadas de acordo com o valor lido. Podem ser usados os modos de endereçamento indireto, absoluto, deslocamento, pré-decremento e pós incremento. *inb* (*input byte*) opera somente com os 8 bits menos significativos de *DATA* e *REG*. Estas instruções são privilegiadas e somente podem ser usadas se a condição *I* estiver ligada. Exemplo:

```
inb   r0, (r5+)
```

Saída de dados

```
out    REG, DATA
outb   REG, DATA
```

Escrever dados (*output*) de *REG* no dispositivo conectado à porta *DATA*. Nenhuma condição é modificada. Podem ser usados os modos de endereçamento indireto, absoluto, deslocamento, pré-decremento e pós incremento. *outb* (*output byte*) opera somente com os 8 bits menos significativos de *DATA* e *REG*. Estas instruções são privilegiadas e somente podem ser usadas se a condição *I* estiver ligada. Exemplo:

```
out    r3, (10)
```

Leitura e modificação atômica

```
swap  REG, DATA
swapb REG, DATA
```

Troca valores entre *REG* e *DATA*, não sendo possível interromper o processador no meio da operação. As condições N e Z são sinalizadas de acordo com o valor transferido de *DATA* para *REG*. Esta instrução é usada para implementação de exclusão mútua, mesmo com multiprocessadores. Exemplo:

```
swap  r1, (1000)
```

Instruções de Controle da Execução

Desvio absoluto

```
jmp    DATA
jmpc   COND, DATA
```

Causa um salto (*jump*) para o endereço absoluto *DATA*, no qual se pode usar qualquer modo de endereçamento. *jmpc* (*jump on condition*) causa um desvio apenas se a condição *COND* for satisfeita. Exemplo:

```
jmp    r2
```

Desvio relativo

```
bra    DATA
brac   COND, IM6
braq   IM10
```

Causa um desvio para um endereço relativo ao *ip* corrente. Desvio zero significa desvio para a próxima instrução. *bra* (*branch*) causa um desvio incondicional, adicionando *DATA* ao *ip*, podendo usar todos os modos de endereçamento. *brac* (*branch on condition*) causa um desvio se a condição *COND* for satisfeita, adicionando o valor imediato *IM₆* ao *ip*. *braq* (*branch quick-immediate*) causa um desvio incondicional adicionando o valor imediato *im₁₀* ao *ip*. Se *DATA*, *IM₁₀* ou *IM₆* for um símbolo, o montador deve calcular a diferença entre o seu valor e o *ip* corrente, avisando se esta diferença extrapolar os limites de 10 e 6 bits respectivamente. Exemplos:

```
bra    10      ; salta para 5 palavras adiante
braq   ali     ; desvia para rótulo "ali"
```

Salta próximo desvio

```
skip   COND
```

Causa o salto (*skip*) de 4 bytes, usualmente uma instrução do tipo *jmp* ou *call* com modo imediato de endereçamento. Equivale a um *brac COND, 4*. Exemplos:

```
skip   nz      ; ---+
jmp     label   ;    |
ld      r0, 5   ; <--+
```

Condições para desvio

O processador Mancha reconhece 13 diferentes condições para desvio nas instruções `brac` e `skip`. Estas condições são alteradas através das instruções de movimentação de dados e de aritmética e lógica. As condições existentes e seus códigos binários correspondentes se encontram na tabela 2. Muitas operações indicam o resultado da comparação entre os dois operandos, *REG* e *DATA*.

Tabela 2 Condições para desvio

nome	número	descrição
Z	0000	resultado foi zero
EQ	0000	$REG = DATA$
C	0001	houve transbordo (<i>carry</i>)
LO	0001	$REG < DATA$ (sem sinal)
O	0010	houve perda de bits (<i>overflow</i>)
N	0011	resultado é negativo
NP	0011	resultado não é positivo
NZ	0100	resultado não é zero
NE	0100	$REG \neq DATA$
NC	0101	não houve transbordo (<i>no carry</i>)
HS	0101	$REG \geq DATA$ (sem sinal)
NO	0110	não houve perda de bits (<i>no overflow</i>)
P	0111	resultado é positivo
GE	1000	$REG \geq DATA$ (N nxor O)
LT	1001	$REG < DATA$ (N xor O)
GT	1010	$REG > DATA$ (NZ and GE)
LE	1011	$REG \leq DATA$ (Z or LT)
HI	1100	$REG > DATA$ (sem sinal, NZ and NC)
LS	1101	$REG \leq DATA$ (sem sinal, Z or C)

Chamada de procedimento

```
call DATA
```

Causa um desvio para o endereço absoluto *DATA*, no qual se pode usar qualquer modo de endereçamento. O endereço da instrução seguinte à instrução `call` (que seria o próximo valor de *ip* caso não houvesse a chamada) é salvo na pilha antes do desvio. Como em um `push`, o ponteiro da pilha é decrementado antes de salvar o endereço. Usado em conjunto com a instrução `ret`. Exemplo:

```
call printf
```

Retorno de procedimento

```
ret
```

Causa um desvio para o endereço absoluto armazenado na pilha (*return form call*). Usado em conjunto com a instrução `call`. O ponteiro da pilha é incrementado de dois depois de recuperar o endereço de retorno (novo valor de *ip*) da memória. As condições N e Z são modificadas.

Causa exceção

`trap IM6`

Causa uma exceção no processador, também conhecida como interrupção causada por software, ou chamada do supervisor. São buscados novos valores para `ip`, `sp`, `cs` e `ds` em uma tabela de exceções e os valores anteriores de `ip`, `sp` e dos registradores de supervisor de `s0` a `s7` são salvos, nesta ordem, na nova pilha. Os bits `S`, `I` e `D` de `sr` são ligados após esta instrução, entrando automaticamente em modo supervisor. Atualmente podem ser usados os valores de 8 a 15 em `IM6`. Usado em conjunto com a instrução `rete`. Exemplo:

```
trap 14
```

Retorno de exceção

`rete`

Causa um desvio para o endereço absoluto armazenado na pilha (*return from exception*). Esta instrução desempilha os registradores `ip`, `sp` e os registradores de supervisor, salvos na pilha quando da instrução `trap`. Esta instrução é uma instrução privilegiada e somente pode ser usada se o bit `S` do registrador `sr` estiver ligado. Esta instrução deve ser usada em conjunto com a instrução `trap`, bem como para o retorno de interrupções geradas pelo hardware.

Desabilita Interrupções

`di`

Causa a ligação do bit `D` do registrador `sr`, que inibe o tratamento de toda e qualquer interrupção (*disable interrupts*). Esta instrução é uma instrução privilegiada e somente pode ser usada se o bit `S` do registrador `sr` estiver ligado.

Habilita Interrupções

`ei`

Causa o desligamento do bit `D` do registrador `sr`, que habilita o tratamento de interrupções (*enable interrupts*). Esta instrução é uma instrução privilegiada e somente pode ser usada se o bit `S` do registrador `sr` estiver ligado.

Parada de execução

`halt`

Causa a parada da execução. Mais nenhuma instrução é buscada após sua execução. Esta instrução é uma instrução privilegiada e somente pode ser usada se o bit `S` do registrador `sr` estiver ligado.

Instruções com Operações Aritméticas

Adição

`add REG, DATA`

`addb REG, DATA`

```
addc  REG, DATA
addq  REG, IM10
```

Soma valor *DATA* ou *IM₁₀* em um registrador *REG*. Os bits N, O, Z, e C do registrador *sr* são modificados de acordo com o resultado. *add*, *addb* (*add byte*) e *addc* (*add carry*) podem usar todos os modos de endereçamento em *DATA*. *addb* opera somente os 8 bits menos significativos de *DATA* e *REG*. *addc* tem a mesma função do *add*, apenas somando também o bit C do registrador *sr*, para permitir a soma de números de mais de 16 bits. *addq* aceita somente um valor imediato *IM₁₀*, de no máximo 10 bits. Exemplos:

```
add    r0, (r5) ; indireto via r5
addb   r1, r2
addq   sp, 5
```

Subtração

```
sub    REG, DATA
subb   REG, DATA
```

Subtrai o valor *DATA* de um registrador *REG*. Os bits N, O, Z, e C do registrador *sr* são modificados de acordo com o resultado. Podem usar todos os modos de endereçamento em *DATA*. *subb* opera somente os 8 bits menos significativos de *DATA* e *REG*. Exemplo:

```
subb   r2, '0'
```

Comparação aritmética

```
cmp    REG, DATA
cmpb   REG, DATA
```

Efetua uma comparação aritmética entre conteúdo do registrador *REG* com o valor *DATA*. Os bits N, O, Z, e C do registrador *sr* são modificados de acordo com o resultado. Podem usar todos os modos de endereçamento em *DATA*. *cmpb* opera somente os 8 bits menos significativos de *DATA* e *REG*. Exemplo:

```
cmpb   r1, r2 ; compara (8 bits) de r1 com r2
```

Multiplificação

```
mul    REG, DATA
```

Multiplica o valor contido no registrador *REG* pelo valor *DATA*, deixando o resultado no registrador. Os bits N, O, Z, e C do registrador *sr* são modificados de acordo com o resultado. Podem usar todos os modos de endereçamento em *DATA*. Exemplo:

```
mul    r1, r5
```

Divisão

```
div    REG, DATA
```

Divide o valor contido no registrador *REG* pelo valor *DATA*, deixando o resultado no registrador. Os bits N, O, Z, e C do registrador *sr* são modificados de acordo com o resultado. Podem usar todos os modos de endereçamento em *DATA*. Exemplo:

```
div    r1, 7
```

Instruções com Operações Lógicas

E lógico

```
and    REG, DATA
```

```
andb   REG, DATA
```

Efetua E (*and*) lógico entre o valor contido no registrador *REG* e o valor *DATA*, deixando o resultado no registrador. Os bits N, O, Z, e C do registrador *sr* são modificados de acordo com o resultado. Podem usar todos os modos de endereçamento em *DATA*. *andb* opera somente os 8 bits menos significativos de *DATA* e *REG*. Exemplo:

```
andb   r1, (r2+) ; ind. pós incrementado via r2
```

Ou lógico

```
or     REG, DATA
```

```
or     REG, DATA
```

Efetua OU (*or*) lógico entre o valor contido no registrador *REG* e o valor *DATA*, deixando o resultado no registrador. Os bits N, O, Z, e C do registrador *sr* são modificados de acordo com o resultado. Podem usar todos os modos de endereçamento em *DATA*. *orb* opera somente os 8 bits menos significativos de *DATA* e *REG*. Exemplo:

```
or     r1, (-r0)
```

Ou-exclusivo lógico

```
xor    REG, DATA
```

```
xorb   REG, DATA
```

Efetua OU-exclusivo (*exclusive-or*) lógico entre o valor contido no registrador *REG* e o valor *DATA*, deixando o resultado no registrador. Os bits N, O, Z, e C do registrador *sr* são modificados de acordo com o resultado. Podem usar todos os modos de endereçamento em *DATA*. *xorb* opera somente os 8 bits menos significativos de *DATA* e *REG*. Exemplo:

```
xor    r1, 255
```


Deslocamento à esquerda

```
shl    REG, DATA  
shlb   REG, DATA
```

Efetua um deslocamento à esquerda (*shift left*) do valor contido no registrador *REG* de *DATA* bits, deixando o resultado no registrador. Os bits N e Z do registrador *sr* são modificados de acordo com o resultado, o bit Z contém o último bit deslocado á esquerda e o bit O é ligado se algum bit for perdido na operação. Podem usar todos os modos de endereçamento em *DATA*. *shlb* opera somente os 8 bits menos significativos de *DATA* e *REG*. Exemplo:

```
shl    r1, 2
```

Deslocamento à direita

```
shr    REG, DATA  
shrb   REG, DATA
```

Efetua um deslocamento à direita (*shift right*) do valor contido no registrador *REG* de *DATA* bits, deixando o resultado no registrador. Os bits N e Z do registrador *sr* são modificados de acordo com o resultado, o bit Z contém o último bit deslocado á direita e o bit O é ligado se algum bit for perdido na operação. Podem usar todos os modos de endereçamento em *DATA*. *shrb* opera somente os 8 bits menos significativos de *DATA* e *REG*. Exemplo:

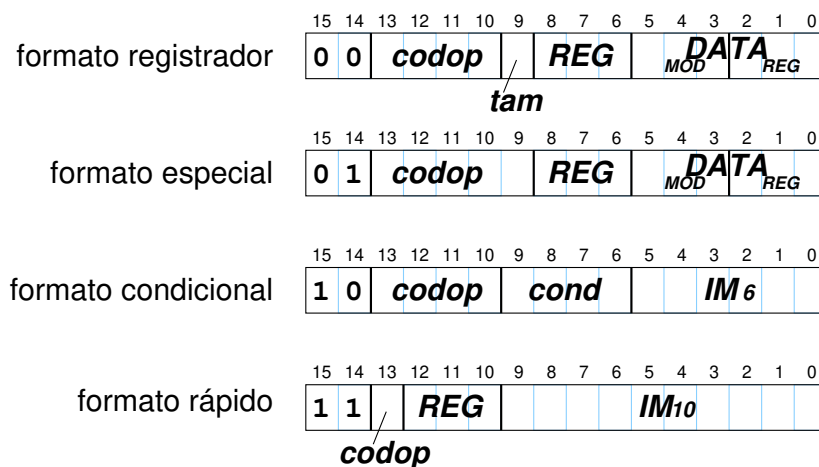
```
shr    r1, r2
```

6 Formato das Instruções

As instruções do processador Mancha se dividem em quatro grupos, segundo o formato binário de sua codificação. Existem quatro formatos de instrução: registrador, especial, condicional e rápido, que podem ser observados na figura 9. As instruções de formato registrador e especial usam os modos de endereçamento definidos na seção 4.

As instruções do Mancha possuem 16 bits, ou seja, são formadas por um código do tamanho de uma palavra. O Mancha é um processador *big-endian*. Ao buscar palavras de instruções na memória, a parte mais significativa (bits 15 a 8) é retirada do endereço da instrução e a parte menos significativa (bits 7 a 0) vem do endereço seguinte.

Figura 9 Formato das instruções



Os dois bits de mais alta ordem da instrução definem o formato da mesma (são quatro os formatos). Quando alguma instrução usar o modo de endereçamento imediato ou o modo absoluto ela possuirá uma palavra adicional de 16 bits. Esta palavra é usada para compor o dado imediato requerido pelo modo de endereçamento.

Instruções do formato registrador (00)

Seguem o formato registrador (formato 00) as instruções de movimentação de dados e de operações aritméticas e lógicas. Todas estas instruções usam modos de endereçamento e podem operar com dois tamanhos de dado (byte e palavra). Os 16 bits da palavra de instrução no formato registrador são divididos em (ver figura 9):

- bits 15-14: 00
- bits 13-10, *codop*: código da micro-operação
- bit 9, *tam*: tamanho da operação, 0:palavra 1:byte
- bits 8-6, *REG*: registrador da operação
- bits 5-0, *DATA*: segundo valor da operação, dividido em:
 - bits 5-3, *MOD*: modo de endereçamento
 - bits 2-0, *REG*: registrador associado ao modo de endereçamento

As instruções do formato registrador se encontram na tabela 3. Existem entretanto algumas instruções do formato registrador que não seguem a mesma notação das outras. Elas operam com registradores e modos de endereçamento pré-determinados. Por exemplo, a instrução `jmp` é uma instrução que modifica diretamente o registrador `ip`. A tabela 4 mostra estas instruções juntamente com sua codificação.

Tabela 3 Instruções no formato registrador

instrução	codop	sintaxe	semântica
<i>load</i>	0000	<code>ld REG, DATA</code>	$REG = DATA$
<i>store</i>	0001	<code>st REG, DATA</code>	$DATA = REG$
<i>add</i>	0010	<code>add REG, DATA</code>	$REG + = DATA$
<i>subtract</i>	0011	<code>sub REG, DATA</code>	$REG - = DATA$
<i>shift left</i>	0110	<code>shl REG, DATA</code>	$REG \ll = DATA$
<i>shift right</i>	0111	<code>shr REG, DATA</code>	$REG \gg = DATA$
<i>compare</i>	1000	<code>cmp REG, DATA</code>	$REG == DATA$
<i>and</i>	1001	<code>and REG, DATA</code>	$REG \text{ and } = DATA$
<i>or</i>	1010	<code>or REG, DATA</code>	$REG \text{ or } = DATA$
<i>exclusive-or</i>	1011	<code>xor REG, DATA</code>	$REG \text{ xor } = DATA$
<i>input</i>	1100	<code>in REG, DATA</code>	$REG = \text{input}(DATA)$
<i>output</i>	1101	<code>out REG, DATA</code>	$\text{output}(DATA) = REG$
<i>swap</i>	1110	<code>swap REG, DATA</code>	$REG < - > DATA$

Tabela 4 Instruções alternativas no formato registrador

instrução	código	sintaxe	equivalente a
<i>jump</i>	00.0000.0.111.mmm.ddd	<code>jmp DATA</code>	<code>ld ip, DATA</code>
<i>branch</i>	00.0010.0.111.mmm.ddd	<code>bra DATA</code>	<code>add ip, DATA</code>
<i>return</i>	00.0000.0.111.010.110	<code>ret</code>	<code>ld ip, (sp+)</code>
<i>pop</i>	00.0000.0.rrr.010.110	<code>pop REG</code>	<code>ld REG, (sp+)</code>
<i>push</i>	00.0001.0.rrr.011.110	<code>push REG</code>	<code>st REG, (-sp)</code>

Instruções do formato especial (01)

Seguem o formato especial (formato 01) as instruções de movimentação de registradores do supervisor, bem como algumas operações de controle da execução. Todas estas instruções usam modos de endereçamento e podem operar somente com dados do tamanho de uma palavra. Os 16 bits da palavra de instrução no formato especial são divididos em (ver figura 9):

- bits 15-14: 01
- bits 13-9, *codop*: código da micro-operação
- bits 8-6, *REG*: registrador da operação (na instrução `call`, usar o valor 000)
- bits 5-0, *DATA*: segundo valor da operação, dividido em:
 - bits 5-3, *MOD*: modo de endereçamento
 - bits 2-0, *REG*: registrador associado ao modo de endereçamento

As instruções do formato especial se encontram na tabela 5. Existem entretanto algumas instruções do formato especial que não seguem a mesma notação das outras. Elas operam com registradores e modos de endereçamento pré-determinados. A tabela 6 mostra estas instruções juntamente com sua codificação.

Tabela 5 Instruções no formato especial

instrução	codop	sintaxe	semântica
<i>call</i>	00000	<i>call DATA</i>	<i>push ip; ip=DATA</i>
<i>load supervisor</i>	00010	<i>lds SUP, DATA</i>	<i>SUP = DATA</i>
<i>store supervisor</i>	00011	<i>sts SUP, DATA</i>	<i>DATA = SUP</i>
<i>multiply</i>	01000	<i>mul REG, DATA</i>	<i>REG * = DATA</i>
<i>divide</i>	01001	<i>div REG, DATA</i>	<i>REG / = DATA</i>
<i>add carry</i>	01010	<i>addc REG, DATA</i>	<i>REG + = DATA + C</i>

Tabela 6 Instruções alternativas no formato especial

instrução	código	sintaxe	equivalente a
<i>pop supervisor</i>	01.00010.rrr.010.110	<i>pops SUP</i>	<i>lds SUP, (sp+)</i>
<i>push supervisor</i>	01.00011.rrr.011.110	<i>pushs SUP</i>	<i>sts SUP, (-sp)</i>

Instruções do formato condicional (10)

As instruções de desvio condicional e algumas instruções de controle do processador seguem o formato condicional (formato 10). Nenhuma destas instruções usa modos de endereçamento. Os 16 bits da palavra de instrução no formato condicional são divididos em (ver figura 9):

- bits 15-14: 10
- bits 13-10, *codop*: código da micro-operação
- bits 9-6, *COND*: o código da condição (somente usado na instrução *brac*, nas outras é usado o valor 0000)
- bits 5-0, *IM₆*: valor imediato de 6 bits

As instruções do formato condicional se encontram na tabela 7. Existem entretanto uma instrução do formato condicional que não segue a mesma notação das outras. Ela possui um *IM₆* pré-determinado. A tabela 8 mostra esta instrução juntamente com sua codificação.

Tabela 7 Instruções no formato condicional

instrução	codop	sintaxe	semântica
<i>trap</i>	0000	<i>trap IM₆</i>	<i>interrupção programada</i>
<i>branch on condition</i>	0001	<i>brac COND, IM₆</i>	<i>se COND ip=ip+IM₆</i>
<i>return from exception</i>	0010	<i>rete</i>	<i>retorna da exceção</i>
<i>halt execution</i>	0011	<i>halt</i>	<i>parada total</i>
<i>disable interrupts</i>	0100	<i>di</i>	<i>sr[D] = 1</i>
<i>enable interrupts</i>	0101	<i>ei</i>	<i>sr[D] = 0</i>
<i>jump on condition</i>	1000	<i>jmpc COND, DATA</i>	<i>se COND ip=DATA</i>

Instruções do formato rápido (11)

Apenas duas instruções, que usam um valor imediato de 10 bits, seguem o formato rápido (formato 10). Nenhuma destas instruções usa modos de endereçamento. Os 16 bits da palavra de instrução no formato rápido são divididos em (ver figura 9):

- bits 15-14: 11
- bits 13, *codop*: código da micro-operação

Tabela 8 Instrução alternativa no formato condicional

instrução	código	sintaxe	equivalente a
<i>skip next jump</i>	10.0001.cccc.000100	skip <i>COND</i>	brac <i>COND</i> , 4

- bits 12-10, *REG*: registrador da operação
- bits 9-0, *IM₁₀*: valor imediato de 10 bits

As instruções do formato rápido se encontram na tabela 9. Existem entretanto instruções do formato rápido que não seguem a mesma notação das outras, porque possuem um registrador pré-determinado. A tabela 10 mostra estas instruções juntamente com sua codificação.

Tabela 9 Instruções no formato rápido

instrução	codop	sintaxe	semântica
<i>quick load</i>	0	ldq <i>REG</i> , <i>IM₁₀</i>	<i>REG</i> = <i>IM₁₀</i>
<i>quick add</i>	1	addq <i>REG</i> , <i>IM₁₀</i>	<i>REG</i> += <i>IM₁₀</i>

Tabela 10 Instruções alternativas no formato rápido

instrução	código	sintaxe	equivalente a
<i>quick jump</i>	11.0.111.iiiiiiiiiii	jmpq <i>IM₁₀</i>	ld ip, <i>IM₁₀</i>
<i>quick branch</i>	11.1.111.iiiiiiiiiii	braq <i>IM₁₀</i>	add ip, <i>IM₁₀</i>

7 Interrupções

Todo processador moderno possui algum sistema de tratamento de interrupções. Este sistema é criado para dar a possibilidade de o processador interromper a execução de um programa qualquer para executar uma sequência urgente de instruções, voltando a executar o programa logo após. Normalmente uma interrupção é relacionada a operações de entrada e saída ou a condições críticas internas ao próprio processador. No primeiro caso, o processador precisa deixar de lado o que está fazendo para atender algum dispositivo de entrada e saída que não pode esperar. No segundo caso, a execução do programa não pode continuar e tem que ser interrompida por algum motivo grave.

Existem diversos tipos de interrupção e elas são numeradas para se diferenciar uma da outra. Existem interrupções geradas por hardware e por software, e ainda, as interrupções geradas por hardware podem ser internas ou externas. As **interrupções internas**, geradas pelo próprio processador, são numeradas de 0 a 4 e podem ser causadas pelos motivos expressos na tabela 11. As interrupções 5, 6 e 7 são reservadas para uso futuro.

As interrupções externas e as geradas por software são numeradas de 8 a 15. As **interrupções externas** são acionadas pela ativação de um pino de interrupção do barramento externo do processador. Normalmente quem aciona este pino é algum dispositivo de entrada ou saída que está conectado ao processador. As **interrupções geradas por software** são aquelas causadas pela execução da instrução `trap`.

Tabela 11 Quadros do vetor de interrupções

número do quadro	endereço do quadro	causa da interrupção
0	0000	início de operação
1	0008	violação de segmento
2	0010	instrução privilegiada
3	0018	divisão por zero
4	0020	instrução ilegal
5	0028	ausência de quadro
:	:	:
8	0040	trap 8 ou interrupção externa
9	0044	trap 9 ou interrupção externa
:	:	:
15	0078	trap 15 ou interrupção externa

Vetor de Interrupções

Quando o processador é interrompido ele precisa mudar o conteúdo do seu registrador `ip` de forma a iniciar a execução de uma sequência de instruções diferente daquela que ele vinha executando. Na verdade, vários registradores precisam ser modificados para que este desvio se concretize de forma correta. Além disso a execução do programa interrompido deve poder ser reiniciada, como se a interrupção não tivesse acontecido.

Para que cada interrupção possa ser tratada por uma sequência de instruções diferente, o processador Mancha possui um **vetor de interrupções**. Este vetor é uma tabela que possui, para cada tipo de instrução, os valores que os registradores do processador devem receber. O conjunto de

valores de registradores de cada interrupção é chamado de **quadro de desvio**.

Cada quadro de desvio é composto por quatro valores de dois bytes, a serem carregados em quatro registradores diferentes: `ip`, `sp`, `cs` e `ds`. Se em um dado quadro o valor a ser carregado em `ip` for zero, quando a interrupção correspondente acontecer ela será ignorada. O vetor de interrupções deve ser armazenado a partir do endereço 0000 da memória física do processador. Ele deve ter 16 quadros de desvio, um para cada interrupção possível, perfazendo assim um total de 128 bytes de memória, que não podem ser usados para outra coisa.

Interrupção de Início de Operação

No momento do início da operação do processador, ele se comporta como se uma interrupção tivesse acontecido. Neste instante são carregados os valores iniciais para `ip`, `sp`, `cs` e `ds`. Portanto, para que o processador Mancha inicie a sua execução corretamente, valores coerentes para estes registradores devem estar previamente armazenados a partir do endereço 0000.

Tratamento das Interrupções

A cada vez que ocorre uma interrupção, o processador entra automaticamente em modo supervisor, salvando em seguida seus registradores em uma pilha. Novos valores são atribuídos a alguns registradores, de forma que o processador passe a executar uma rotina previamente associada à interrupção. Esta rotina deve tomar providências para tratar o evento causador da interrupção. Por exemplo, se um programa em execução causar uma violação de segmento, por acessar endereços fora dos limites permitidos, a rotina de tratamento de interrupção deve abortar a execução do programa.

No início do tratamento de cada interrupção, o processador busca automaticamente na memória novos valores para `ip`, `sp`, `cs` e `ds` no quadro de desvio correspondente à interrupção que está ocorrendo. Os valores anteriores de todos os registradores são salvos na nova pilha (`r0` fica no topo da pilha, `s7` aparece no fundo da pilha). Os bits `S`, `I` e `D` de `sr` são ligados antes mesmo da operação de salvamento, entrando automaticamente em modo supervisor, com entrada e saída permitida e interrupções desabilitadas. O retorno do código de tratamento de uma interrupção é feito com a instrução `rete`, que desempilha todos os registradores salvos automaticamente na pilha no início da interrupção.

8 Organização da Memória

Do ponto de vista de uma instrução em execução, um endereço de memória do Mancha possui 16 bits. Isso dá às instruções de máquina a possibilidade de acessar 65536 endereços diferentes. Cada endereço de memória corresponde a um byte. Para armazenar dados maiores que um byte, mais de uma posição de memória deve ser usada (e em consequência, mais de um endereço). O conjunto de 65536 endereços diferentes é chamado de **espaço de endereçamento** do processador.

O Mancha é um processador do tipo *big-endian*, ou seja, quando uma palavra (16 bits) é armazenada na memória, isto se faz começando pelo seu byte de mais alta ordem, seguido pelo byte de menor ordem. Por exemplo, para armazenar o valor hexadecimal A21C no endereço 0350 de memória, o valor A2 é escrito no endereço 0350 e o valor 1C é escrito no endereço 0351.

Os acessos à memória são divididos em dois tipos: **acessos a instruções** e **acessos a dados**. Cada vez que uma instrução for ser executada, ela será previamente buscada da memória, na posição dada pelo registrador *ip*. Cada vez que um modo de endereçamento de memória for usado em uma instrução qualquer, um dado será lido ou escrito na memória, usando o endereço proveniente do modo de endereçamento. Estas duas modalidades de acesso — busca de instruções e acesso a dados — são os dois tipos de acesso existentes e eles se fazem em dois espaços de endereçamentos diferentes.

Cada acesso à memória ocorre em um certo endereço, seja ele dado pelo *ip*, no caso de uma busca de instrução, ou pelo modo de endereçamento, no caso de um acesso a dados. Entretanto, este endereço de acesso não é diretamente passado para a memória. Ele sofre uma transformação, que pode ser feita por segmentação ou por paginação. Assim, o endereço usado pela instrução, dentro do processador, é chamado de **endereço virtual**. O endereço passado à memória é chamado de **endereço real**. Diz-se também que o espaço de endereçamento de um processador é um espaço de **memória virtual**.

Segmentação

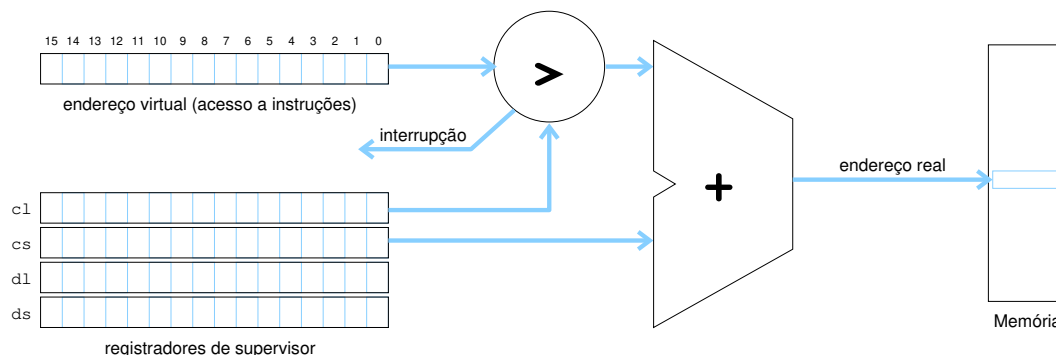
A segmentação é o modo normal de operação do processador Mancha. Neste modo, a transformação feita nos endereços de memória é um simples deslocamento em relação ao início da memória. Para implementar a segmentação, o processador usa os registradores de supervisor de *s4* a *s7*, atribuindo-lhes novos nomes: *cs*, base do segmento de código, *c1*, tamanho do segmento de código, *ds*, base do segmento de dados, e *d1*, tamanho do segmento de dados. *cs* e *c1* são usados nos acessos a instruções e *ds* e *d1* nos acessos a dados.

O endereço real é calculado pela soma do endereço virtual com a base do segmento correspondente. Entretanto, um acesso a um endereço virtual superior ao tamanho do segmento é considerado um erro de programação. Este acesso não se concretiza e causa no processador uma interrupção de violação de segmento. Este mecanismo permite manter diversos programas em memória, sem necessidade de relocação. O espaço de endereçamento de cada programa fica limitado à região de memória que o diz respeito. A figura 10 ilustra o cálculo do endereço real a partir de um endereço virtual de um acesso a instruções.

Se o bit *S* do registrador *sr* estiver ligado, o processador se encontra em modo supervisor.

Neste caso são desconsiderados os limites dos segmentos. Um acesso fora dos limites não causa nenhuma interrupção. Outra maneira de fazer com que o processador desconsidere os limites dos segmentos, mesmo em modo usuário, é atribuir valores 0000 aos registradores de limite de segmento.

Figura 10 Acesso à memória usando segmentação



Paginação

A paginação é um modo de memória virtual um pouco mais sofisticado que a segmentação. Enquanto que a segmentação define somente duas fatias de memória (uma para instruções e outra para dados), a paginação permite atribuir centenas de diferentes trechos de memória a um dado programa. Por outro lado, se a segmentação permite dividir a memória em pedaços de qualquer tamanho, a paginação trabalha com pedaços pequenos e de tamanho fixo.

O modo de paginação é acionado ao ligar o bit P do registrador s.r. Neste modo, o registrador de supervisor s4 passa a ser chamado de pt, ponteiro da tabela de páginas, ficando os registradores s5 a s7 sem função específica definida.

Os espaços de endereçamento (de instruções e de dados) são divididos em **páginas**, de tamanho fixo de 512 bytes. Assim, como cada espaço possui 65536 endereços, eles são divididos em 128 páginas cada, perfazendo um total de 256 páginas de memória. Isso equivale a dizer que um programa em execução com paginação pode usar 256 páginas diferentes de memória.

Da mesma forma que os espaços de endereçamento são divididos em páginas, a memória real é dividida em quadros de 512 bytes. A associação de quadros a páginas se dá através de uma tabela, chamada **tabela de páginas**. Como um programa tem 256 páginas em seu espaço de endereçamento, uma tabela de páginas precisa ter 256 entradas. Assim, a tabela é do mesmo tamanho que um quadro de memória, contendo 256 entradas de 16 bits. Cada entrada da tabela contém um número de quadro diferente. Cada programa em execução deve possuir uma tabela de páginas, apontando para os quadros de memória que este programa tem direito de acessar.

O cálculo do endereço real dos acessos à memória se dá por uma tradução de endereços, fazendo uso dos números de quadros contidos na tabela de páginas. Para cada acesso à memória, calcula-se inicialmente o número da página que se está acessando. Esta cálculo é simples: basta dividir o endereço acessado pelo tamanho de uma página. Como o tamanho da página é uma potência de dois, a divisão por ele é desnecessária.

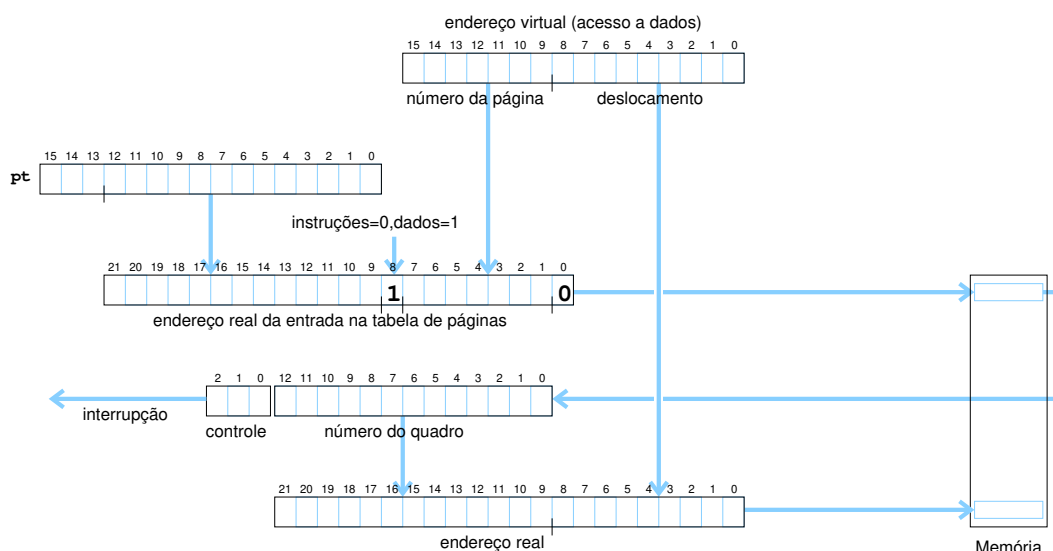
O **número da página** se calcula tomando os 7 bits mais significativos do endereço. Os 9 bits menos significativos do endereço são chamados de **deslocamento** e são guardados para uso pos-

terior. Se for um acesso a dados, ao número da página soma-se 128 (liga-se o oitavo bit), de forma que as páginas de instruções sejam numeradas de 0 a 127 e as páginas de dados sejam numeradas de 128 a 255.

O número da página é usado como índice na tabela de páginas, de onde se retira uma palavra de 16 bits. Os 3 bits mais significativos desta palavra são chamados **bits de controle de acesso** e são removidos para uso posterior. O **número do quadro** possui treze bits, podendo identificar 8192 quadros diferentes, o que limita a memória real em 4 megabytes. O número do quadro é multiplicado por 512 para encontrar-se o endereço real do primeiro byte do quadro na memória real. Para multiplicar um número por 512 basta deslocá-lo nove bits à esquerda.

Como o endereço real do primeiro byte do quadro, pode-se agora calcular o endereço do byte acessado na memória. Esse cálculo se faz somando o endereço do primeiro byte do quadro com o deslocamento obtido no desmembramento do endereço virtual. A figura 11 resume os passos para o cálculo do endereço real a partir de um endereço virtual.

Figura 11 Acesso à memória usando paginação



O controle de acesso à memória é feito através dos três bits de controle contidos nas entradas da tabela de páginas. Os dois bits mais significativos determinam o tipo de acesso que o programa tem a cada uma de suas páginas. O tipo de acesso pode ser 00, ausente, 01, leitura, 10, escrita, ou 11, modificado. O terceiro bit, menos significativo, é chamado de bit de acesso e serve somente para indicar que houve um acesso (de qualquer tipo) ao respectivo quadro.

O tipo ausente significa que a entrada não contém um número de quadro e que os endereços do intervalo correspondente não podem ser acessados. Um acesso a um quadro ausente causa uma interrupção de ausência de quadro no processador. O tipo leitura significa que o programa pode ler os dados contidos na página correspondente, mas não pode escrever. Uma tentativa de escrita causa uma interrupção do tipo violação de segmento. Os tipos escrita e modificado permitem leitura e escrita no quadro correspondente.

Todo quadro que tiver permissão de escrita deve ser inserido na tabela com o tipo 10 (escrita). Cada vez que um quadro do tipo escrita receber um acesso de escrita ele é promovido ao tipo modificado. Da mesma forma, o bit de acesso deve ser inserido desligado na tabela, pois ele será ligado no primeiro acesso que houver a este quadro. Estas marcações servem para indicar

ao sistema operacional que o quadro em questão, respectivamente, teve seu conteúdo modificado e foi recentemente acessado. Isto permite a implementação de memória virtual com auxílio de memória secundária (geralmente disco). Em caso de falta de espaço em memória real, o sistema operacional pode decidir que alguns quadros da memória real serão gravados em memória secundária, para uso posterior. O sistema pode, por exemplo, decidir que os quadros menos recentemente usados (terceiro bit desligado) é que devem ser guardados em memória secundária.

9 Resumo do Processador Mancha

Tabela 12 Instruções de Usuário

instrução	operandos	código da operação
add	<i>REG, DATA</i>	00.0010.0.rrr.mmm.ddd
addb	<i>REG, DATA</i>	00.0010.1.rrr.mmm.ddd
addc	<i>REG, DATA</i>	01.01010.rrr.mmm.ddd
addq	<i>REG, IM₁₀</i>	11.1.rrr.iiiiiiiiiii
and	<i>REG, DATA</i>	00.1001.0.rrr.mmm.ddd
andb	<i>REG, DATA</i>	00.1001.1.rrr.mmm.ddd
bra	<i>DATA</i>	00.0010.0.111.mmm.ddd
brac	<i>COND, IM₆</i>	10.0001.cccc.iiiiiii
braq	<i>IM₁₀</i>	11.1.111.iiiiiiiiiii
call	<i>DATA</i>	01.00000.000.mmm.ddd
cmp	<i>REG, DATA</i>	00.1000.0.rrr.mmm.ddd
cmpb	<i>REG, DATA</i>	00.1000.1.rrr.mmm.ddd
div	<i>REG, DATA</i>	01.01001.rrr.mmm.ddd
jmp	<i>DATA</i>	00.0000.0.111.mmm.ddd
jmpc	<i>COND, DATA</i>	10.1000.cccc.mmm.ddd
ld	<i>REG, DATA</i>	00.0000.0.rrr.mmm.ddd
ldb	<i>REG, DATA</i>	00.0000.1.rrr.mmm.ddd
ldq	<i>REG, IM₁₀</i>	11.1.rrr.iiiiiiiiiii
mul	<i>REG, DATA</i>	01.01000.rrr.mmm.ddd
or	<i>REG, DATA</i>	00.1010.0.rrr.mmm.ddd
orb	<i>REG, DATA</i>	00.1010.1.rrr.mmm.ddd
pop	<i>REG</i>	00.0000.0.rrr.010.110
push	<i>REG</i>	00.0001.0.rrr.011.110
pushs	<i>SUP</i>	01.00011.rrr.011.110
ret		00.0000.0.111.010.110
shl	<i>REG, DATA</i>	00.0110.0.rrr.mmm.ddd
shlb	<i>REG, DATA</i>	00.0110.1.rrr.mmm.ddd
shr	<i>REG, DATA</i>	00.0111.0.rrr.mmm.ddd
shrb	<i>REG, DATA</i>	00.0111.1.rrr.mmm.ddd
skip	<i>COND</i>	10.0001.cccc.000100
swap	<i>REG, DATA</i>	00.1110.0.rrr.mmm.ddd
st	<i>REG, DATA</i>	00.0001.0.rrr.mmm.ddd
stb	<i>REG, DATA</i>	00.0001.1.rrr.mmm.ddd
sts	<i>SUP, DATA</i>	01.00011.sss.mmm.ddd
sub	<i>REG, DATA</i>	00.0011.0.rrr.mmm.ddd
subb	<i>REG, DATA</i>	00.0011.1.rrr.mmm.ddd
trap	<i>IM₆</i>	10.0000.0000.iiiiiii
xor	<i>REG, DATA</i>	00.1011.0.rrr.mmm.ddd
xorb	<i>REG, DATA</i>	00.1011.1.rrr.mmm.ddd

Tabela 13 Instruções Privilegiadas

instrução	operandos	código da operação
di		10.0100.0000.000000
ei		10.0101.0000.000000
halt		10.0011.0000.000000
in	<i>REG, DATA</i>	00.1100.0.rrr.mmm.ddd
inb	<i>REG, DATA</i>	00.1100.1.rrr.mmm.ddd
lds	<i>SUP, DATA</i>	01.00010.sss.mmm.ddd
out	<i>REG, DATA</i>	00.1101.0.rrr.mmm.ddd
outb	<i>REG, DATA</i>	00.1101.1.rrr.mmm.ddd
pops	<i>SUP</i>	01.00010.rrr.010.110
rete		10.0010.0000.000000

Tabela 14 Códigos de Registrador

<i>rrr sss ddd</i>	<i>REG</i>	<i>SUP</i>
000	r0	s0 sr
001	r1	s1
010	r2	s2
011	r3	s3
100	r4	s4 cs pt
101	r5 bp	s5 cl
110	r6 sp	s6 ds
111	r7 ip	s7 dl

Tabela 15 Códigos de Modo de Endereçamento

<i>mmm</i>	<i>DATA</i>
000	<i>REG</i>
001	<i>(REG)</i>
010	<i>(REG+)</i>
011	<i>(-REG)</i>
100	<i>IMED</i>
101	<i>(IMED)</i>
110	<i>(REG+IMED)</i>

Tabela 16 Tipos de Valores Imediatos

tipo	descrição	limites
<i>IM₆</i>	valor imediato de 6 bits com sinal	-32 ... 31
<i>IM₁₀</i>	valor imediato de 10 bits com sinal	-512 ... 511
<i>IMED</i>	valor imediato de 16 bits (palavra adicional)	com sinal -32768 ... 32767
		sem sinal 0 ... 65535

10 Montador

Foi implementado um montador para o processador Mancha, capaz de gerar código objeto para arquivos contendo instruções e pseudo-instruções Mancha. As instruções reconhecidas por este montador são aquelas descritas nas seções 5 e 6. As pseudo-instruções reconhecidas pelo montador do Mancha são descritas a seguir.

Linha de comando

Para executar o montador do Mancha, deve-se usar a seguinte linha de comando:

```
prompt% mas [opções] [-O listagem] [-o saída] [entrada]
```

Os termos entre colchetes são opcionais (os colchetes não devem ser digitados). As *opções* indicam as seguintes ordens ao montador:

- -V: Indicar a sua versão;
- -P: Passar o arquivo de entrada pelo pré-processador (cpp) antes de iniciar a montagem;
- -S: Listar a tabela de símbolos;
- -L: Listar na saída padrão o resultado da montagem no formato compatível com o Mancha Mínimo;
- -C: Listar na saída padrão o resultado da montagem junto como código fonte;
- -O *listagem*: Usar arquivo *listagem* em vez da saída padrão, nas opções -L ou -C;
- -o *saída*: Gravar o objeto (resultado da montagem) no arquivo *saída* (geralmente com a extensão .o), usar - para especificar a saída padrão;
- *entrada*: Usar arquivo *entrada* como arquivo fonte (geralmente com a extensão .s), se for omitido é usada a entrada padrão.

Quando não for especificado arquivo fonte nem objeto, são usadas entrada e saída padrão. Quando for especificado somente o fonte, o objeto será um arquivo com o mesmo nome, trocando a extensão para .o.

Módulos, rótulos e relocação

Os arquivos de entrada do montador são escritos em linguagem de montagem do processador Mancha. Cada arquivo contém um conjunto inter-relacionado de instruções e dados e é chamado de **módulo**. Assim, um arquivo de entrada do montador é um **módulo fonte** e um arquivo de saída do montador é um **módulo objeto**.

Por razões de engenharia de software, módulos fonte devem poder ser escritos de forma independente uns dos outros. Assim, o programador pode dotá-los de interfaces bem definidas e reutilizá-los quando for escrever aplicações. Por exemplo, um módulo que contenha a implementação das funções seno e cosseno deve poder ser usado por um programa de computação gráfica, bem como por um programa de matemática. Para isso, os módulos são inicialmente escritos de forma independente uns dos outros. Posteriormente, usando-se um programa ligador, diversos módulos são concatenados para criar-se um programa executável.

Devido à possibilidade de concatenação de um conjunto qualquer de módulos, parte-se do pressuposto que não se sabe qual trecho de memória os conterá no momento da execução. Assim,

não é possível o programador saber de antemão qual será o endereço de cada instrução e de cada dado na memória. Isso se configura em um problema, pois a maioria das instruções que causam o desvio da execução do processador e a maioria das instruções que operam com dados precisam de um argumento que diga com qual endereço operar. Uma instrução de desvio geralmente contém o endereço da próxima instrução. Uma instrução de operação com dados geralmente possui o endereço do dado. Sem saber a posição de memória de instruções e dados é praticamente impossível escrever um programa.

Para solucionar o problema do desconhecimento dos endereços de dados e instruções são usados dois artifícios: endereços relativos e rótulos. Cada módulo é então montado como se o endereço de sua primeira instrução ou dado fosse zero. Cada vez que o montador precisar gravar no módulo objeto um endereço dele mesmo ele grava também uma **marca de relocação**. Esta marca indica que o endereço deve ser modificado se o módulo não for estar na posição para a qual ela foi gerada. Assim, um endereço gravado no módulo objeto é um **endereço relativo** ao início do módulo.

Como exemplo de relocação, supor que o programador deseje desviar a execução para a quarta instrução de um módulo, e que as três primeiras instruções do módulo ocupem no total 8 bytes. Como o módulo é gerado como se estivesse na posição zero, a instrução a ser usada é `jmp 0008`. Entretanto, quando este módulo tiver sido ligado com muitos outros e for ser executado, suponha que ele venha a estar na posição 1000 de memória. Nesta situação, a instrução que desvia para a quarta instrução é `jmp 1008`, ou seja, o endereço 0008 deve ser relocado para 1008. Por esta razão este endereço é marcado como relocável pelo montador.

Para evitar a necessidade de o programador contar o tamanho de cada instrução para saber a posição de cada uma das subseqüentes, o montador usa uma variável, chamada de **endereço de carga corrente**. Esta variável é incrementada a cada vez que uma instrução é gerada no módulo objeto. Assim, o montador sempre sabe para que endereço uma dada instrução está sendo montada. Quando uma dada instrução precisar ser referenciada (para um desvio, por exemplo, o programador pode associar a instrução a um nome, chamado de **rótulo**). Ao perceber a associação de um rótulo a uma instrução, o montador guarda o rótulo e seu valor — o endereço de carga corrente — em uma tabela, chamada **tabela de símbolos**. Cada referência subseqüente ao rótulo causa a geração de seu valor no código objeto.

Segmentos

A maioria dos processadores modernos apresenta algum tipo de mecanismo de controle de acesso à memória. Estes mecanismos servem principalmente para evitar que um programa (errado ou malicioso) modifique áreas de memória que não lhe dizem respeito. Em geral pode-se dizer ao processador que uma certa região da memória contém os dados e que ele não deve modificar nenhuma informação fora desta região. Deve-se também informar a região onde se encontram as instruções, de forma que o processador possa buscar nela as instruções a serem executadas (notar que a região de instruções não pode se modificada, somente lida).

Os programas são sempre compostos por vários módulos, cada um deles contendo dados e instruções. Um programa executável possui então muitos trechos com instruções e muitos trechos com dados. Por esta razão, dados e instruções são gravados separadamente e os módulos objeto são divididos em seções, ou **segmentos**. Nos formatos modernos de arquivos objeto existem seções separadas para diversas utilidades diferentes. O formato objeto do Mancha possui

três segmentos: segmento de instruções (`text`), segmento de dados (`data`), e segmento de dados não inicializados (`bss`).

O programador deve selecionar, no programa fonte, onde começa o código de cada um dos segmentos. Existem pseudo-instruções para trocar o segmento a ser usado. Enquanto nenhum segmento for especificado, o montador gera código para o segmento de instruções. Dentro do módulo objeto é gravado o segmento de instruções seguido do segmento de dados. O segmento de dados não inicializados não é gravado, pois este serve apenas para denotar um espaço a ser reservado para dados, sem nenhum valor inicial. Por esta razão, no segmento de dados não inicializados somente podem haver definições de espaço (ver *define storage* a seguir) e de símbolos (rótulos).

Linguagem de Montagem

Cada módulo fonte é composto de diversas instruções e diversas descrições de dados, todas elas dispostas em linhas. Cada linha contém duas partes, ambas opcionais: um **rótulo** e uma **instrução**. O caracter ':' serve para identificar um rótulo na linha, sendo este apostro ao final do seu nome. A instrução pode ser uma instrução da linguagem de montagem do processador Mancha ou uma pseudo-instrução do montador.

A figura 12 contém um exemplo de módulo fonte em linguagem de montagem do Mancha. A primeira linha do exemplo contém uma pseudo-instrução do montador (`.text`). A segunda linha possui o rótulo `func` e uma instrução do tipo `load`. A terceira linha somente contém uma instrução `call`. No final da seção do montador se encontra a figura 13, com um exemplo mais completo.

Figura 12 Pequeno exemplo de módulo fonte

```
.text
func:  ld      r0, 5
       call   printf
```

Símbolos

O montador do Mancha possui um mecanismo de substituição de nomes textuais por valores numéricos. Isto é necessário para a implementação de rótulos, mas serve também para facilitar a programação. Assim, o montador possui uma tabela de símbolos que contém, para cada símbolo, um nome e um valor. Quando um símbolo da tabela for usado em uma instrução, seu nome é substituído pelo seu valor numérico.

Cada símbolo somente pode ser definido uma vez. Existem duas formas de se definir um novo símbolo:

- Em um rótulo, que define um símbolo cujo valor é o endereço de carga corrente;
- Pela pseudo-instrução *equate*, que define um símbolo com um valor qualquer.

O nome de um símbolo pode conter letras maiúsculas e minúsculas, números e os caracteres '\$', '.', e '_', mas não pode começar por um número. Letras maiúsculas e letras minúsculas são letras diferentes nos nomes dos símbolos (`AQUI` e `aqui` são símbolos diferentes). O símbolo \$ é um símbolo especial, que representa o endereço de carga corrente.

Para poder relacionar um módulo com o outro, é necessário o uso de símbolos públicos e externos. Os **símbolos públicos** são símbolos definidos dentro de um módulo que podem ser usados em outros módulos. Ao utilizar um símbolo de outro módulo, este deve ser declarado como **símbolo externo**. Quando um símbolo externo for usado, nenhum código será gerado e uma lacuna será deixada no módulo objeto. Esta lacuna somente será preenchida quando se souber o valor do símbolo, que será definido em outro módulo.

De todos os símbolos existentes na tabela de símbolos durante a montagem, apenas os símbolos públicos e externos são gravados nos módulos objeto. Os símbolos públicos marcam quais endereços do módulo podem ser usados por outros módulos. Os símbolos externos, por outro lado, marcam as lacunas deixadas no código. O ligador, ao concatenar os módulos objeto, irá calcular o valor final de cada símbolo (após a relocação) e irá preencher as lacunas deixadas.

Valores Imediatos

Diversas instruções do processador Mancha contém um valor numérico em seu interior. Algumas possuem um valor imediato de 6 bits, denominado *IM₆*, outras possuem um valor imediato de 10 bits, chamado de *IM₁₀*. Muitas instruções usam um modo de endereçamento que, se for imediato, absoluto ou deslocamento (ver seção 4), possuem seu valor imediato em uma palavra adicional de 16 bits, chamado de *IMED*.

Se o código fonte inclui valores imediatos numéricos, estes são considerados absolutos (não-relocáveis). Para marcar explicitamente a relocação de um valor imediato, podem ser usados prefixos com o nome do segmento de relocação (por exemplo `text : 50`).

Todos os tipos de valores imediatos (*IMED*, *IM₆* e *IM₁₀*) podem ser definidos por símbolos. O código gerado com valores imediatos do tipo *IMED* definidos por um rótulo é marcado como sendo relocável, com relação ao segmento no qual foi definido. Por outro lado, o código gerado com valores imediatos do tipo *IM₆* e *IM₁₀* não é marcado como relocável.

Existem instruções de desvio que possuem um valor imediato relativo ao endereço corrente, como `bra` ou `brac`. Ao encontrar valores numéricos, o montador não os muda e gera o código numérico encontrado no programa fonte. Ao encontrar símbolos, o montador subtrai seus valores do endereço corrente, de forma a causar o desvio para o rótulo especificado.

Pseudo-instruções

As pseudo-instruções descritas a seguir utilizam a seguinte notação: o texto em tamanho *fixo* designa um nome de pseudo-instrução e o texto em *itálico* designam uma opção com vários valores válidos. As opções existentes são:

- *nome*: nome de um símbolo, ex.: `main aqui $` ;
- *número*: valor inteiro absoluto, ex.: `0 122 0xFF08` ;
- *carac*: caracter, ex.: `'a' '%' '='` ;
- *reloc*: base de relocação, ex.: `text data bss` ;
- *cadeia*: sequência de caracteres, ex.: `"exemplo"` ;
- *numrel*: *reloc:número*, ex.: `data:0x100 text:0` ;
- *valor*: *nome*, *número*, *numrel* ou *carac*;
- *byte*: *cadeia*, *número* ou *carac*, ex.: `"Tab" '5' 250` ;

- *soma*: *valor* + *número*;
- *difer*: *valor* – *valor*;
- *expr*: *valor*, *soma* ou *difer*.

Atribuição `.equ` *Equate*

A pseudo-instrução `.equ` é usada para atribuir um valor a um símbolo. Sintaxe:

`.equ nome = expr`

Origem `.org` *Origin*

Usa-se a pseudo-instrução `.org` para mudar o endereço de carga corrente. Sintaxe:

`.org valor`

Define byte `.db`

A pseudo-instrução `.db` define bytes a serem carregados no segmento corrente. É usada para preencher a memória com bytes de dados inicializados. Sintaxe:

`.db byte [, byte, ... ]`

Define palavra `.dw` *Define word*

A pseudo-instrução `.dw` define palavras a serem carregadas no segmento corrente. É usada para preencher a memória com palavras de dados inicializados. Sintaxe:

`.dw valor [, valor, ... ]`

Define espaço `.ds` *Define space*

A pseudo-instrução `.ds` define um espaço sem código no segmento corrente. É usada para deixar espaço na memória para dados não inicializados. Sintaxe:

`.ds número`

Externo `.ext` *Extern*

A pseudo-instrução `.ext` declara um símbolo como sendo externo. Sintaxe:

`.ext nome [, nome, ... ]`

Público `.pub` *Public*

A pseudo-instrução `.pub` declara um símbolo como sendo público. Sintaxe:

`.pub nome [, nome, ... ]`

Segmento de instruções `.text` *Text*

A pseudo-instrução `.text` indica ao montador que as instruções que se seguem devem ser carregadas no segmento de instruções. Sintaxe:

`.text`

Segmento de dados inicializados `.text` *Data*

A pseudo-instrução `.data` indica ao montador que as instruções que seuem devem ser carregadas no segmento de dados inicializados. Sintaxe:

```
.data
```

Segmento de dados não inicializados `.bss` *Blank storage space*

A pseudo-instrução `.text` indica ao montador que as instruções que seuem devem ser carregadas no segmento de dados não inicializados. Sintaxe:

```
.bss
```

Informação de depuração `.dbg` *Debug information*

A pseudo-instrução `.dbg` causa a geração de registros de informação na área de depuração do módulo objeto. Cada registro tem um tipo de 1 byte, um símbolo associado, um número e uma cadeia de caracteres terminada com nulo. Sintaxe:

```
.dbg carac, nome, número, cadeia
```

Figura 13 Exemplo de programa em linguagem de montagem

; Código de montagem Mancha gerado pelo lcc

```
.text
.pub    main
main:
    push    bp
    ld      bp, sp
    sub     sp, 22
    call    _console_init
    ld      r0, L.2      ; r0=&global
    push    r0
    call    puts
    addq    sp, 2
    ldq     r0, 0        ; const
    st      r0, (bp-22)  ; i
    jmp     L.6
L.3:
    ld      r0, (bp-22)  ; i
    shl     r0, 1
    ld      r1, (bp+6)   ; argv
    add     r0, r1
    ld      r0, (r0)
    push    r0
    call    puts
    addq    sp, 2
L.4:
    ld      r0, bp
    addq    r0, -22      ; r0=&i
    ld      r1, (r0)
```

```

        add    r1, 1
        st     r1, (r0)

L.6:    ld     r0, (bp-22) ; i
        ld     r1, (bp+4)  ; argc
        cmp    r0, r1
        jmpc   lt, L.3
        ldq    r0, 20      ; const
        push   r0
        ld     r0, bp
        addq   r0, -20      ; r0=&str
        push   r0
        call   getsn
        addq   sp, 4
        ld     r0, bp
        addq   r0, -20      ; r0=&str
        push   r0
        call   puts
        addq   sp, 2
        ldq    r0, 0        ; const

L.1:    addq   sp, 22
        pop    bp
        ret                                ; main

        .ext   _console_init
        .ext   puts, getsn

        .data   ; LIT

L.2:    .db    "exemplo", 0

```

11 Programas Utilitários

Alguns programas utilitários servem a manipular os arquivos objetos Mancha.

Editor de Ligações

O editor de ligações, ou simplesmente ligador, é um programa que concatena módulos objeto, resolvendo referências externas. O editor de ligações do Mancha reconhece os arquivos objetos gerados pelo montador e gera arquivos de saída no mesmo formato.

Para executar o editor de ligações do Mancha, deve-se usar a seguinte linha de comando:

```
prompt% mld [opções] -o saída entrada [...] [-l biblioteca]
```

Os termos entre colchetes são opcionais e os termos com reticências podem ser repetidos (colchetes e reticências não devem ser digitados). As *opções* indicam as seguintes ordens ao ligador:

- -V: Indicar a sua versão;
- -v: Aumentar a quantidade de mensagens de informação (verbosidade). Pode haver mais de uma opção -v;
- -S: Listar a tabela de símbolos;
- -k: Gerar módulo executável não relocável, concatenando todos os segmentos no segmento de instruções;
- -i: Ligar de forma incremental, mantendo na saída as tabelas de símbolos externos e públicos;
- -o *saída*: Gravar o executável (resultado da ligação) no arquivo *saída* (geralmente sem extensão), usar - para especificar a saída padrão;
- -l *biblioteca*: Varrer arquivo(s) *biblioteca* ... como biblioteca(s) de arquivos objeto (geralmente com a extensão .a), no intuito de resolver referências externas pendentes. Se algum arquivo for encontrado, acrescentar este arquivo à ligação. Pode haver mais de uma opção -l;
- *entrada* [...]: Ligar este(s) arquivo(s) objeto (geralmente com a extensão .o).

Listar Tamanho Tamanho de Módulos Objetos Binários

Para executar, deve-se usar a seguinte linha de comando:

```
prompt% msize opções objeto
```

O utilitário *msize* lê o cabeçalho de módulos objetos binários e relata os valores nele encontrados. As *opções* são opcionais e indicam as seguintes ordens:

- -V: indicar a versão do programa *msize*;
- -x: dar informações em hexadecimal;
- -o: dar informações em octal;
- -f: listar todas as informações.

Converter Objeto Mancha Mínimo em Objeto Mancha

Para executar, deve-se usar a seguinte linha de comando:

```
prompt% menc < mínimo > objeto
```

O utilitário `menc` converte o arquivo encontrado na entrada padrão, no formato de codificação do Mancha Mínimo, em um arquivo objeto codificado no formato do Mancha completo. A saída padrão é usada para gerar o arquivo objeto de saída.

Converter Objeto Mancha em Objeto Mancha Mínimo

Para executar, deve-se usar a seguinte linha de comando:

```
prompt% mdec < objeto > mínimo
```

O utilitário `menc` converte o arquivo encontrado na entrada padrão, no formato de codificação do Mancha completo, em um arquivo objeto codificado no formato do Mancha Mínimo. A saída padrão é usada para gerar o arquivo objeto de saída.

12 Formato do arquivo objeto

Arquivos objeto são arquivos que contém códigos resultantes da compilação de um determinado pedaço de um programa. Comumente, usa-se um arquivo fonte diferente para cada módulo independente de um programa a ser compilado. Quando um procedimento usa funções que não são definidas no próprio arquivo fonte, estas funções são chamadas de funções externas. Assim, um arquivo objeto contém referências a endereços que não estão definidos nele, pois os arquivos fonte de um programa podem ser compilados de forma independente entre si. Basta que se tenha um programa que faça a concatenação de diversos arquivos objeto, de forma que todas as referências a endereços externos possam ser resolvidas, isto é, as suas definições tenham sido encontradas em outro arquivo. O resultado da ligação de diversos arquivos objeto geralmente é um arquivo pronto para ser executado, sem nenhuma referência externa. Este tipo de arquivo é chamado de arquivo executável.

A principal diferença entre um arquivo objeto e um arquivo executável é que um arquivo objeto pode conter referências a endereços que serão definidos em outros arquivos. Já um arquivo executável geralmente não tem referências a endereços fora dele. Na realidade, com o uso da ligação dinâmica, a ligação dos arquivos objeto passou a ser adiada até o momento da carga do programa na memória. Assim estes passaram a ser ligados com bibliotecas compartilhadas entre diversos programas em execução. Conseqüentemente, arquivos executáveis passaram a ter referências a endereços de fora dele, e assim não há mais diferença prática entre eles e os arquivos objeto.

Como regra geral, há que se definir um tipo diferente de arquivo objeto para cada arquitetura de processador diferente. A razão disso é que cada arquitetura de processador possui instruções e formatos próprios para representar endereços de memória. Houve muita evolução no sentido de definir formatos padronizados de arquivos objeto, como os arquivos COFF [1] ou os arquivos ELF [2]. A maioria dos sistemas operacionais modernos usa algum tipo derivado destes formatos padronizados para representar seus arquivos objeto.

O formato de arquivo objeto ou executável do Mancha é derivado do formato inicialmente usado no sistema Unix [3]. Foi assim escolhido por ser um formato bastante simples a ser compreendido e implementado, ainda que capaz de representar todas as informações relevantes de um arquivo objeto comum. O arquivo objeto do Mancha é então composto das seguintes áreas:

- Cabecalho,
- Carga do segmento de instruções,
- Carga do segmento de dados inicializados,
- Relocação do segmento de instruções,
- Relocação do segmento de dados,
- Símbolos públicos,
- Símbolos externos,
- Informações.

Cada uma destas áreas é descrita a seguir.

Área de cabeçalho

A área de cabeçalho é composta de 8 palavras de 2 bytes, como segue:

- palavra 0: Assinatura contendo o valor BACA em hexadecimal;
- palavra 1: Tamanho em bytes da área de carga do segmento de instruções;
- palavra 2: Tamanho em bytes da área de carga do segmento de dados inicializados;
- palavra 3: Tamanho em bytes da área de carga do segmento de dados não inicializados;
- palavra 4: Tamanho em bytes da área de símbolos públicos;
- palavra 5: Tamanho em bytes da área de símbolos externos;
- palavra 6: Tamanho em bytes da área de informações;
- palavra 7: reserva;

Lembrar que o Mancha é *big-endian*, ou seja, ao armazenar palavras o byte mais significativo é armazenado em primeiro lugar. Por exemplo, na assinatura (palavra 0), o primeiro byte contém o valor hexadecimal BA e o segundo contém CA.

Áreas de carga

As áreas de carga dos segmentos de instruções e de dados inicializados são compostas dos bytes dos segmentos a serem carregados. O tamanho de cada uma destas áreas está definido no cabeçalho. Por razões evidentes, não existem bytes de carga para o segmento dados não inicializados.

Áreas de relocações

As áreas de relocações dos segmentos de instruções e de dados inicializados são compostas dos bytes de relocação dos segmentos a serem carregados. O tamanho de cada uma destas áreas pode ser calculado a partir do tamanho da sua área de carga correspondente. Cada byte de relocação contém informações de relocação de quatro palavras da área de carga, sendo dois bits para cada palavra.

A relocação é feita por palavras. Cada palavra da área de carga pode ser marcada como sendo relocável ou não. Entretanto, somente o primeiro byte de cada palavra relocável é marcado. Por exemplo, se o endereço 100 estiver marcado como sendo relocável relativo ao segmento de instruções, a palavra armazenada nos endereços 100 e 101 será relocada, ou seja, somada com o endereço de base da área de instruções no momento da sua carga.

Para cada byte da área de carga existe um par de bits na área de relocações indicando o seu tipo de relocação. Cada byte de relocação armazena assim informação referente a quatro posições da área de carga. O tamanho da área de relocações é então igual a um quarto do tamanho da área de código, ocupando sempre um número inteiro de bytes. O código usado nos bits que marcam a relocação é o seguinte:

- 00 Não relocar;
- 01 Relocar em relação ao segmento de instruções;
- 10 Relocar em relação ao segmento de dados inicializados;
- 11 Relocar em relação ao segmento de dados não inicializados.

Área de símbolos públicos

A área de símbolos públicos contém os símbolos definidos dentro do arquivo objeto que podem ser referenciados por outros arquivos objeto. Cada símbolo público do arquivo possui uma entrada nesta área.

As entradas de símbolos públicos são formadas pelo nome do símbolo, o número do segmento no qual o símbolo foi definido e o seu endereço no segmento em questão. O nome do símbolo deve ser terminado com um byte de valor zero. Um byte inteiro (apenas 2 bits seriam suficientes) é usado para identificar o segmento no qual o símbolo foi definido, com o valor 1 para o segmento de instruções, 2 para o segmento de dados inicializados e 3 para o segmento de dados não inicializados. O endereço de definição do símbolo ocupa 2 bytes. Desta forma, cada entrada ocupa um número variável de bytes, igual ao número de caracteres ocupados pelo nome do símbolo mais quatro.

Área de símbolos externos

A área de símbolos externos contém uma relação de referências a símbolos externos ao arquivo. Cada símbolo externo referenciado possui somente uma entrada nesta área.

As entradas de referência a símbolos externos são formadas pelo nome do símbolo e por uma lista de segmentos e endereços nos quais o símbolo foi referenciado. O nome do símbolo deve ser terminado com um byte de valor zero. Da mesma forma que para símbolos públicos, um byte é usado para identificar o segmento, usando a mesma codificação. Como um símbolo pode ser referenciado em vários locais diferentes de diferentes segmentos, várias duplas segmento-endereço podem aparecer após o nome do símbolo. O valor zero no lugar do segmento serve como delimitador, devendo ser usado após a última dupla.

Área de informações

A área de informações contém somente informações usadas por um depurador, como por exemplo o tipo dos dados, a correspondência entre bytes de código e a linha de programa fonte que os deu origem, etc. Cada entrada nesta seção é composta de um tipo (um byte), um segmento (um byte) um endereço (dois bytes) um tamanho (dois bytes) e um símbolo terminado com um byte de valor zero.

Exemplo de arquivo objeto

Esta seção descreve um exemplo de arquivo objeto, cujos códigos hexadecimais aparecem na figura 14. Este arquivo foi gerado através da montagem do programa que aparece na figura 13.

A primeira linha do exemplo (endereço 0000) é um cabeçalho, com a assinatura (BA CA), o tamanho do segmento de instruções (00 78), o tamanho do segmento de dados inicializados (00 08), o tamanho do segmento de dados não inicializados (00 00), o tamanho da área de símbolos públicos (00 08), o tamanho da área de símbolos externos (00 2B), e o tamanho da área de informações (00 00). Os bytes dos endereços 0010 a 0088 formam o segmento de instruções e os bytes dos endereços 0088 a 008F compõem o segmento de dados inicializados. Os códigos aí contidos são resultado da montagem de instruções e pseudo-instruções. Estes códigos não serão explicados aqui mas podem ser facilmente verificados servindo-se das descrições encontradas

nas seções iniciais deste texto.

Figura 14 Exemplo de arquivo objeto

```
0000: BA CA 00 78 00 08 00 00 00 08 00 2B 00 00 00 00
0010: 05 5E 01 46 0D A0 00 16 40 20 00 00 00 20 00 00
0020: 04 1E 40 20 00 00 F8 02 C0 00 04 35 FF EA 01 E0
0030: 00 46 00 35 FF EA 18 20 00 01 00 75 00 06 08 01
0040: 00 08 04 1E 40 20 00 00 F8 02 00 05 E3 EA 00 48
0050: 08 60 00 01 04 48 00 35 FF EA 00 75 00 04 20 01
0060: A2 60 00 22 C0 14 04 1E 00 05 E3 EC 04 1E 40 20
0070: 00 00 F8 04 00 05 E3 EC 04 1E 40 20 00 00 F8 02
0080: C0 00 F8 16 01 56 01 D6
0088:                                     65 78 65 6D 70 6C 6F 00
0090: 00 00 00 20 00 00 00 00 01 00 00 00 00 00 00 00
00A0: 00 00 00 00 10 00 00 00 00 00 00 00 00 00 00
00AE:                                     00 00
00B0: 6D 61 69 6E 00 01 00 00
00B8:                                     67 65 74 73 6E 00 01 00
00C0: 60 00 70 75 74 73 00 01 00 6C 01 00 36 01 00 14
00D0: 00 5F 63 6F 6E 73 6F 6C 65 5F 69 6E 69 74 00 01
00E0: 00 0A 00
```

As marcas de relocação do segmento de instruções aparecem de 0090 a 00AD e as marcas do segmento de dados estão em 00AE e 00AF. Nestas duas seções de relocação, as únicas marcas de relocação aparecem na seção de instruções, nos endereços 0093, 0098 e 00A4. Cada marca de relocação ocupa dois bits, assim cada byte contém quatro marcas, sendo numeradas de 0 a 3, da menos significativa à mais significativa. A primeira marca (20 no endereço 0093) vale 10 correspondendo a uma relocação em relação à seção de dados. Como esta marca se encontra na posição 2 do byte 3 (quarto byte da seção), ela se refere ao endereço 000E, que contém uma palavra no valor 0000. Assim, usando o mesmo raciocínio, a marca do endereço 0098 indica uma relocação em relação à seção de instruções, no endereço 0020, ou seja, do valor 0046. Da mesma forma, a marca do endereço 00A4 indica uma relocação em relação à seção de instruções, no endereço 0052, ou seja, do valor 0022.

A área de símbolos públicos se encontra nos bytes com endereços entre 00B0 e 00B7, contendo a declaração de apenas um símbolo público de nome `main` (6D, 61, 69, 6E e 00), definido na seção de instruções (01), no endereço 0000 (00 e 00). A área de símbolos externos aparece entre 00B8 e 00E2, com a definição de três símbolos referenciados. O primeiro símbolo, `gets`, (67, 65, 74, 73, 6E, 00) é referenciado apenas no endereço 0060 da seção de instruções (01, 00, 60, seguido de 00, fim da lista). O segundo símbolo, `puts`, por sua vez, é referenciado nos endereços 006C, 0036 e 0014, todos da seção de instruções. O terceiro e último símbolo referenciado, `_console_init`, possui referência no endereço 000A da seção de instruções.

13 Simulador

Foi implementado um simulador para o processador Mancha, capaz de executar código objeto Mancha. As instruções reconhecidas por este processador são aquelas descritas na documentação do Mancha. A arquitetura deste simulador também está descrita nesta documentação.

Para executar o montador, deve-se usar a seguinte linha de comando:

```
prompt% msim [opções] inicial
```

- -d: executa em modo de depuração;
- -m: indica estatísticas de execução no final;
- -r: emula Mancha Mínimo;
- -D *arquivo*: especifica um arquivo para conter os discos simulados;
- *inicial*: especifica arquivo executável inicial a carregar (*boot*).

Este programa simula a execução do processador Mancha, dando carga prévia em um arquivo executável. Com a opção -D pode-se definir um ou mais arquivos que abrigam os discos controlados pela interface de disco.

Com a opção -d, o simulador entra em modo depurador, aceitando os comando abaixo:

- g: (*go*) dispara a execução;
- s: (*step*) executa uma instrução;
- b: (*breakpoint*) insere/retira pontos de parada;
- r: (*registers*) verifica o conteúdo dos registradores;
- l: (*load*) modifica um registrador;
- d: (*dump*) lista conteúdo da memória;
- u: (*unasm*) lista instruções;

Arquitetura Simulada

Esta seção define a arquitetura de um computador hipotético baseado no Mancha. Esta arquitetura é implementada pelo simulador desenvolvido para este processador. Ao final da seção, as tabelas 17 e 18 apresentam um resumo das portas e interrupções simuladas.

Memória

Um banco de memória volátil de 1Mbytes está diretamente conectado ao processador. Cada byte é endereçado separadamente e o processador faz dois acessos à memória para carregar uma palavra. O simulador permite carregar nesta memória um arquivo objeto qualquer antes de iniciar a simulação.

Console

Existe um terminal de console, que possui dois registradores, um de dados e outro de estado. Na porta 0001 do Mancha está conectado o registrador de dados da interface. Uma saída na porta de dados causa a emissão de um caracter na console, uma entrada na porta de dados causa a leitura de um caracter da console. Na porta 0002 do Mancha está conectado o registrador de estado da console. Este registrador de estado contém dois bits, indicando se um caracter pode ser enviado (bit 0) e se um caracter pode ser recebido (bit 1).

Cada vez que um novo caracter for digitado na console, a interface vai ligar o bit 1 do seu registrador de estado. Quando este caracter for lido do registrador de dados pelo Mancha este bit será desligado. Similarmente, quando a interface estiver pronta para enviar um caracter para a tela da console, ela vai ligar o bit 0 de seu registrador de estado. Quando o Mancha escrever um caracter no registrador de dados da interface, este bit será desligado.

Sempre que a interface ligar um ou dois de seus bits de estado ela causará uma interrupção número 8 no Mancha.

Disco

A interface de disco possui seis registradores de oito bits: operação, face, trilha, setor, endereço alto e endereço baixo. Eles estão ligados às portas 0010 a 0015 do processador. O processador programa uma operação a ser efetuada no disco usando as seis portas.

Os registradores de face, trilha e setor servem para identificar em que trilha, face e setor do disco a operação será efetuada. Eles estão ligados respectivamente nas portas 0010, 0011 e 0012 do processador. O registradores de endereço tem 16 bits e está conectado nas portas 0014 e 0015 (respectivamente, partes alta e baixa, oito bits em cada). Ele serve para armazenar o endereço de memória do processador que deverá ser acessado durante a operação.

O registrador de operação possui 4 bits, dois dos quais (bits 0 e 1) que identificam qual é a unidade de disco a operar e dois bits (bits 6 e 7) que identificam se a operação é uma leitura (00) ou uma escrita (01). Ele está ligado na porta 0013 e deve ser o último registrador a ser escrito.

A seqüência de programação de uma operação é a seguinte:

- escrever o número da face a ser lida ou escrita na porta 0010 (registrador de face);
- escrever o número da trilha a ser lida ou escrita na porta 0011 (registrador de trilha);
- escrever o número do setor a ser lido ou escrito na porta 0012 (registrador de setor);
- escrever uma palavra contendo o endereço de memória a usar na operação nas portas 0014 e 0015 (registrador endereço);
- escrever o número da unidade a ser lida, juntamente com a operação a ser feita na porta 0013 (registrador de operação). Isso dispara o início da operação.

Quando uma operação estiver terminada, a interface causará uma interrupção número 9 no Mancha.

O registrador de operação pode ser lido para obter o estado da interface a qualquer momento. Se o bit 0 estiver ligado (bit de interface ocupada), a interface já está trabalhando em um pedido de

leitura ou escrita e não receberá ordens de escrita em nenhum de seus registradores. Se o bit 1 estiver ligado (bit de indicação de erro), algum problema ocorreu na programação da operação anterior. Este bit é ligado após o início da operação, com a escrita no registrador de operação e desligado com a primeira leitura do registrador de estado. Toda operação, mesmo errada, causa uma interrupção no processador.

A interface efetua a operação acessando diretamente a memória do processador. Uma leitura preenche a memória do processador com os bytes lidos de um setor. Uma escrita lê os bytes da memória do processador e escreve-os em um setor.

Os discos podem ter um número variável de trilhas, faces e setores. Os setores de discos diferentes podem ter tamanhos diferentes. Uma operação do tipo 10 (bits 6 e 7 do registrador de operação) permite recuperar estes tamanhos. São escritos, a partir do endereço de memória dado, quatro palavras: o número de faces, o número de trilhas por face, o número de setores por trilha e o tamanho do setor.

Relógio

A interface de relógio possui dois registradores de 16 bits, chamados contador e limite, ligados às portas 0020 e 0022 respectivamente. Ambos registradores podem ser lidos e escritos a qualquer momento pelo processador. A cada pulso de relógio do processador, o registrador de contador é incrementado. Cada vez que o contador for igual ao limite, a interface causa uma interrupção de número 10 no processador.

Controlador de Interrupções

A interface de controle de interrupções possui um registrador de 8 bits, ligado na porta 0030 do processador. Cada um dos bits deste registrador habilita ou desabilita uma das oito possíveis interrupções externas. O bit 0 corresponde à interrupção 8 e o bit 7 corresponde à interrupção 15.

Tabela 17 Resumo das Portas de Entrada e Saída Simuladas

endereço (hexa)	função
0001	dados da console (entrada e saída)
0002	controle da console, bits [—rs], r=caracter recebido, s=pronto para enviar
0010	programa face do disco
0011	programa trilha do disco
0012	programa setor do disco
0013	escrita: inicia operação, leitura: lê o estado bits da operação: [oo—uu], oo=operação (00=leitura, 01=escrita, 10=identifica disco), uu=unidade de disco bits do estado: [—eb], e=ocorreu erro, b=interface ocupada
0014	programa endereço de DMA (8 bits mais significativos)
0015	programa endereço de DMA (8 bits menos significativos)
0020	contador do relógio (8 bits mais significativos)
0021	contador do relógio (8 bits menos significativos)
0022	limite do relógio (8 bits mais significativos)
0023	limite do relógio (8 bits menos significativos)
0030	controlador de interrupções, bits [fedcba98], f=interrupção 15, e=interrupção 14, ...

Tabela 18 Resumo das Interrupções Simuladas

número	origem
0–7	interrupções internas
8	console
9	interface de disco
10	relógio
11–15	desocupadas

14 Mancha Mínimo

O processador Mancha Mínimo é uma versão simplificada do processador Mancha. Ele somente possui quatro registradores de usuário: *r0*, acumulador, *r1*, operando, *r2*, índice e *ip*, contador de programa. Ele implementa apenas um pequeno subconjunto das instruções do processador Mancha. Ele possui somente dois bits de estado: *Z* e *C* (zero e carry) e não possui interrupções. Ao ser ativado, ele começa a executar o código armazenado a partir do endereço zero. Para maiores explicações sobre o comportamento das instruções, referir-se às seções de descrição do processador Mancha.

Instruções do Mancha Mínimo

A seguir aparecem as instruções reconhecidas pelo processador Mancha Mínimo. A notação usada abaixo é a mesma usada para descrever o processador Mancha.

Carga e armazenamento de registradores

```
ldi  $IM_{10}$ 
ldo
ldx
ldax
stax
```

ldi carrega no acumulador um valor imediato IM_{10} , de 10 bits, com sinal. *ldo* e *ldx* carregam respectivamente os registradores de operando e de índice com o valor armazenado no acumulador. *ldax* e *stax* respectivamente carrega e armazena uma palavra da/na memória cujo endereço é dado pelo registrador de índice. A condição *Z* é ligada se o valor movimentado for igual a zero. Exemplos:

```
ldi 340
stax
```

Entrada e saída de dados

```
in
out
```

in lê (*input*) valores fornecidos pelo dispositivo conectado à porta correspondente ao registrador de índice e armazena-o no acumulador. *out* escreve o valor do acumulador na porta correspondente ao registrador de índice. A condição *Z* é ligada se o valor movimentado for igual a zero. Exemplo:

```
in
```

Desvio e controle da execução

```
jmpq  $IM_{10}$ 
skip COND
nop
halt
```

`jmpq` causa um salto (*jump*) para o endereço absoluto IM_{10} . `skip` salta 4 bytes (2 instruções do Mancha Mínimo) caso uma condição seja verdadeira. As condições reconhecidas são: C e Z (*carry* e *zero*), que correspondem aos bits de estado C e Z estarem ligados. `nop` é uma instrução que não tem nenhum efeito (*no operation*) e `halt` causa a parada do processador. Exemplo:

```
skip z
jmpq 200
```

Aritmética e lógica

```
add
and
or
xor
addx
```

`add`, `and`, `or` e `xor` fazem respectivamente operações de soma, e lógico, ou lógico, ou exclusivo lógico entre o acumulador e o operando, armazenando o resultado no acumulador. `addx` soma o índice ao acumulador. A condição Z é ligada se o valor resultado for igual a zero. Nas operações `add` e `addx`, a condição C é ligada se o resultado for maior que 16 bits. Exemplo:

```
and
```

Formato das Instruções

código binário	instrução	sintaxe	equivalente Mancha
110000.iiiiiiiiiii	load immediate	<code>ldi IM_{10}</code>	<code>ldq r0, IM_{10}</code>
0000000001000000	load operand	<code>ldo</code>	<code>ld r1, r0</code>
0000000010000000	load index	<code>ldx</code>	<code>ld r2, r0</code>
0000000000001010	load indexed	<code>ldax</code>	<code>ld r0, (r2)</code>
0000010000001010	store indexed	<code>stax</code>	<code>st r0, (r2)</code>
0011000000001010	input	<code>in</code>	<code>in r0, (r2)</code>
0011010000001010	output	<code>out</code>	<code>out r0, (r2)</code>
110111.iiiiiiiiiii	jump	<code>jmpq IM_{10}</code>	<code>ldq ip, IM_{10}</code>
100001000.c.000100	skip	<code>skip $COND$</code>	<code>skip $COND$</code>
0000000000000000	no-operation	<code>nop</code>	<code>ld r0, r0</code>
1000110000000000	halt	<code>halt</code>	<code>halt</code>
0000100000000001	add	<code>add</code>	<code>add r0, r1</code>
0010010000000001	and	<code>and</code>	<code>and r0, r1</code>
0010100000000001	or	<code>or</code>	<code>or r0, r1</code>
0010110000000001	exclusive-or	<code>xor</code>	<code>xor r0, r1</code>
0000100000000010	add index	<code>addx</code>	<code>add r0, r2</code>

Condições do skip do Mancha Mínimo

código binário	condição
0	Z
1	C

Figura 15 Exemplo de programa para o Mancha Mínimo

```
; exemplo de programa Mínimo
        .text
        .pub      main
        .ext      fim
main:
        ldi      par1
        ldx
        ldax
        ldo
        ldi      par2
        ldx
        ldax
        add
        ldo
        ldi      result
        ldx
        ldi      0
        or
        stax
        jmpq     fim

        .data
par1:    .dw      5
par2:    .dw      10
result:  .ds      2
```

Arquitetura do Mancha Mínimo

Somente a console está conectada ao Mancha Mínimo. Para maiores informações, referir-se à descrição da arquitetura do Mancha.

A memória do Mancha Mínimo tem somente 1024 bytes. Quando os registradores de índice e contador de programa forem usados como endereços de memória, somente serão usados os seus 10 bits menos significativos.

Formato do arquivo objeto/executável do Mancha Mínimo

O arquivo objeto ou executável do Mancha é um arquivo texto em ASCII, contendo um cabeçalho seguido de linhas de código. O montador do Mancha completo gera este tipo de código com a opção -L.

Cabeçalho O cabeçalho de um arquivo objeto do processador Mancha Mínimo é composto de uma linha, contendo o seguintes caracteres:

```
baca 0000 0000 0000 0000 0000 0000 0000
```

Linhas de Código Após o cabeçalho, um arquivo objeto tem uma sequência não delimitada de linhas, na qual cada linha pode conter uma definição de palavra, uma definição de um símbolo público ou uma definição de símbolo externo. Os endereços e segmentos de carga do código do Mancha Mínimo não precisam obedecer uma ordem crescente. As linhas de definição de símbolo públicos ou externos podem aparecer a qualquer momento no meio do código. Não pode haver mais de uma linha de definição para um mesmo símbolo externo.

Formatos das linhas de código:

- Definição de palavra a ser carregada:

s:xxxx w r:oooo

- *s* é o segmento de carga (1=código, 2=dados)
- *xxxx* é o endereço de carga
- *r* define a relocação (0=absoluto 1=relativo ao código, 2=relativo aos dados)
- *oooo* é o valor da palavra a carregar

- Definição de um símbolo público:

P:nnnnnn s:oooo

- *nnnnnn* é o nome do símbolo
- *s* é o segmento de definição do símbolo
- *oooo* é o valor do símbolo

- Definição de referências a um símbolo externo:

E:nnnnnn s:oooo s:oooo s:oooo ... 0

- *nnnnnn* é o nome do símbolo referenciado
- *s* é o segmento da referência
- *oooo* é o endereço da referência

Montador, pseudo-instruções

O montador do Mancha Mínimo somente é capaz de traduzir um subconjunto dos programas traduzidos pelo montador do Mancha completo. Este montador reconhece somente:

- Instruções e condições do processador Mancha Mínimo;
- Dados imediatos de 10 bits (IM_{10});
- As pseudo-instruções: `.equ`, `.org`, `.dw`, `.ext`, `.pub`, `.text` e `.data`.

Para maiores informações, referir-se à descrição do montador do Mancha.

Figura 16 Exemplo de arquivo objeto para o Mancha Mínimo

```

baca 0000 0000 0000 0000 0000 0000 0000
1:0000 w 2:C000
1:0002 w 0:0080
1:0004 w 0:000A
1:0006 w 0:0040
1:0008 w 2:C002
1:000A w 0:0080
1:000C w 0:000A
1:000E w 0:0801
1:0010 w 0:0040
1:0012 w 0:C004
1:0014 w 0:0080
1:0016 w 0:C000
1:0018 w 0:2801
1:001A w 0:040A
1:001C w 0:DC00
2:0000 w 0:0005
2:0002 w 0:000A
2:0004 w 0:0000
P:main 1:0000
E:fim 1:001C 0
```

Referências

- [1] Inserir referência ao formato COFF.
- [2] Inserir referência ao formato ELF.
- [3] Inserir referência ao formato a.out.